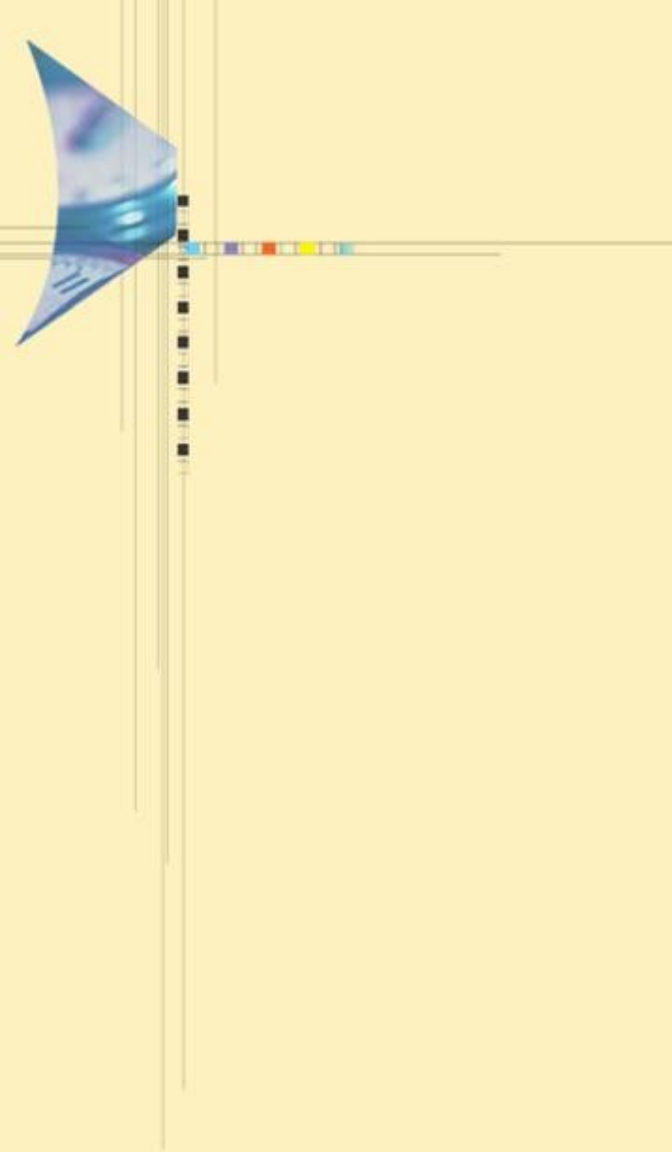




第 8 讲

回溯法

—An Efficient Searching Algorithm



本章内容

- 8.1 引言
 - 8.1.1 简单问题例的算法描述（图的着色问题，皇后问题）
 - 8.1.2 回溯法解的长度分析
- 8.2 回溯策略的算法设计思想及分析方法
 - 8.2.1 适用条件、问题解的表示、约束函数、状态空间树
 - 8.2.2 回溯策略的基本思想
 - 8.2.3 回溯算法一般框架
 - 8.2.4 回溯策略的本质 - 约束穷举



本章内容

- 8.3. 回溯策略的经典应用（可选讲部分）
 - 8.3.1 可满足问题的算法描述、算法分析、程序实现
 - 8.3.2 子集和问题的算法描述、算法分析、程序实现
 - 8.3.3 哈密顿回路问题的算法描述、算法分析、程序实现
 - 8.3.4 0/1 背包问题的算法描述、算法分析、程序实现
 - 8.3.5 货郎担问题问题的算法描述、算法分析、程序实现

8.1 简介

- 在许多真实世界的问题中，像大多数 NP 难问题一样，通过穷尽搜索数量巨大但有限多个可能性，可以获得一个解。
- 事实上对于所有这些问题都不存在用穷尽搜索之外的方法来解决问题的算法。



怎么样开发出系统化的搜索技术？

怎样能够将搜索空间减少到尽可能的小？



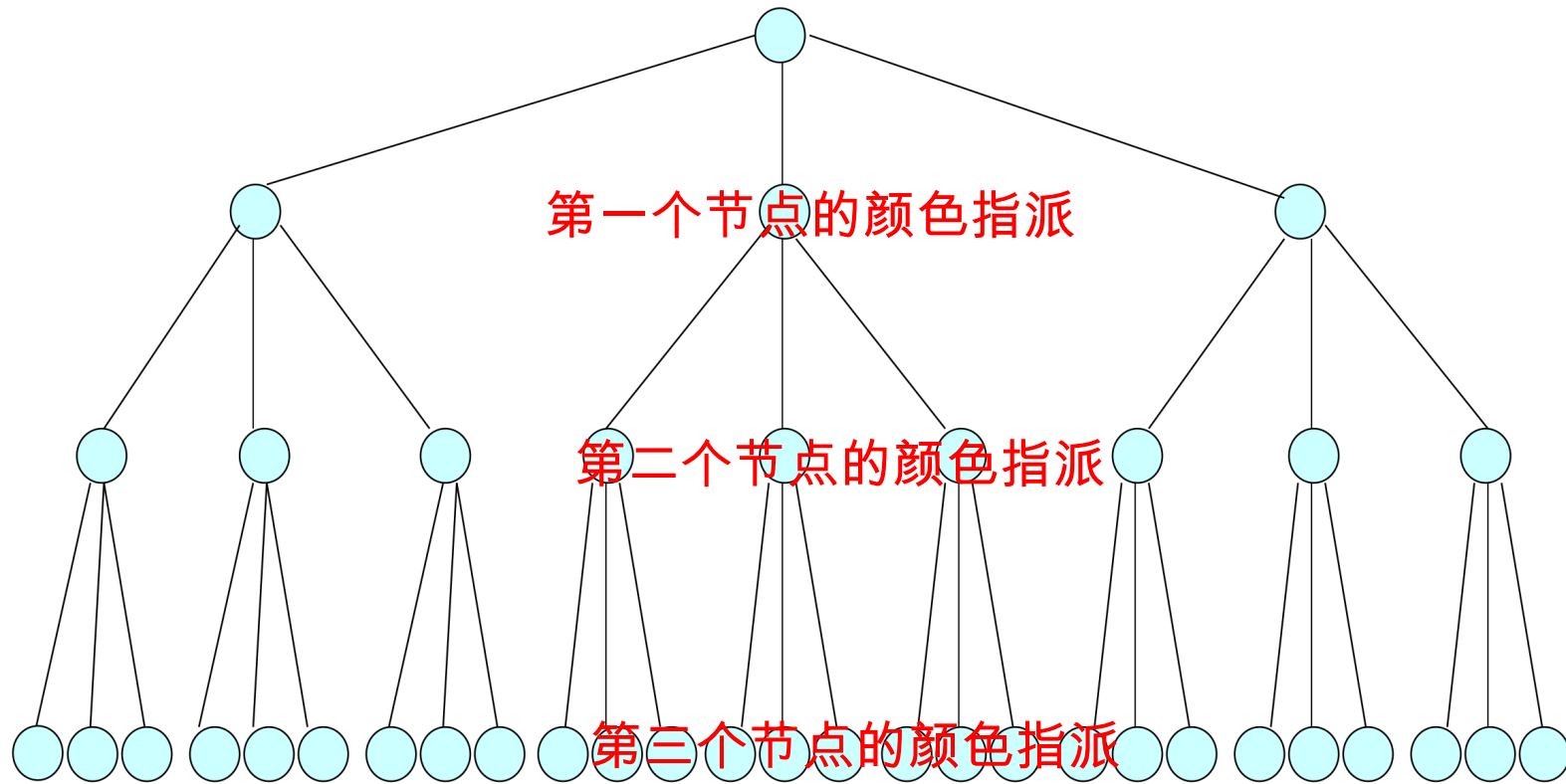
8.1.1 图着色——问题描述

? 问题描述

- 给出一个无向图 $G=(V,E)$ ，需要用三种颜色之一为 v 中的每个顶点着色，三种颜色分别为 1,2 和 3，使得没有两个邻接的顶点有同样的颜色。
- 一种着色可以用 n 元组 $(c_1, c_2, \dots, c_n)$ 来表示，使 $c_i \in \{1, 2, 3\}$ ， $1 \leq i \leq n$ 。
- 例如， $(1,2,2,3,1)$ 表示一个有 5 个顶点的图的着色，一个 n 个顶点的图共有 3^n 种可能的着色（合法的和非法的）。

8.1.1 图着色——搜索树

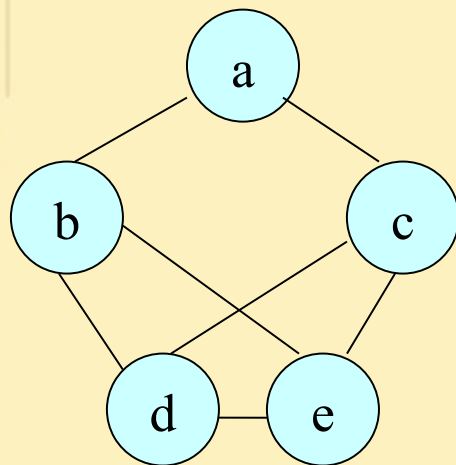
- 所有可能的着色的集合可以用一棵完全的三叉树来表示，称为**搜索树**。在这棵树中，从根到叶节点的每一条路径代表一种**着色指派**。



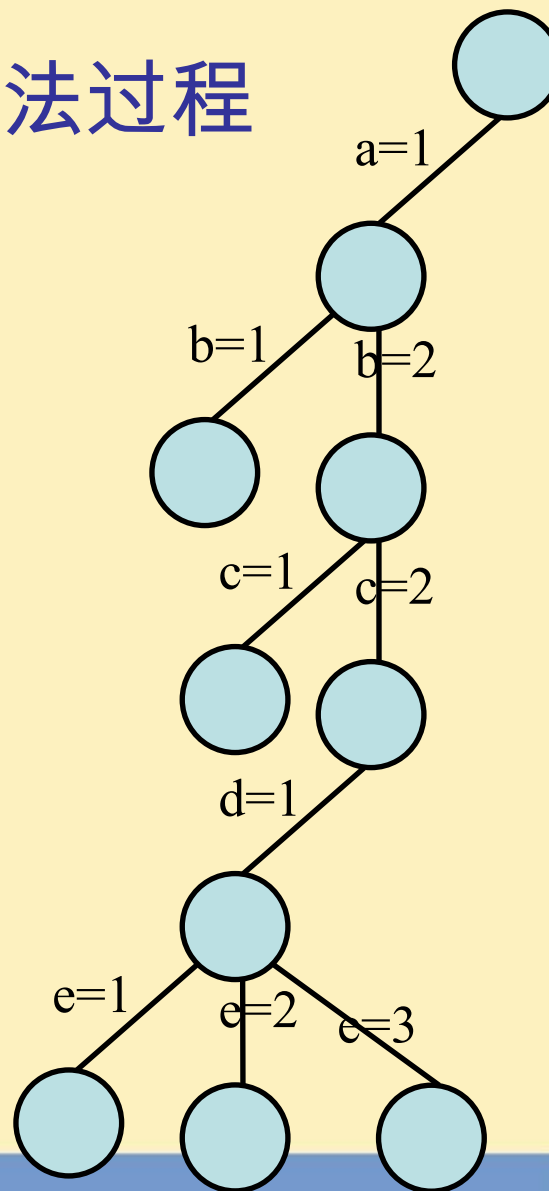
8.1.1 图着色——回溯解法

- 满足条件——两个邻接的着色顶点没有同样的颜色。
图的一个不完全着色称为**部分解**。
- 思路：回溯法通过每次一个节点地生成基底树来工作。
 - 如果从根到当前节点（叶节点）的路径对应于一个合法着色过程就终止（除非找到所有可能的着色方案）
 - 如果这条路径的长度小于 n ，并且相应的着色是部分的，那么就生成现节点的一个子节点，并将它标记为**现节点**。
 - 另一方面，如果对应的路径不是部分的（最后一个节点已经选定一种着色方案），那么现节点标识为**死节点**，并生成对应于另一种颜色的**新节点**。
 - 如果所有三种颜色都已经试过且没有成功，搜索就回溯到父节点，它的颜色被改变，依次类推。

8.1.1 图着色——算法过程



Find a solution!



8.1.1 图着色——算法分析

1. 我们仅生成了由 363 个节点组成的搜索树中 9 个节点后就到达了解。（根节点不算）

$$3+9+27+81+243=363$$

1. 节点是用**深度优先搜索**的方法生成的；
1. 不需要存储整棵搜索树，我们只需要存储根到当前活动节点的路径。事实上，根本没有生成有形的物理节点，整棵树是隐含的。在 3 着色的例子中，我们只需要保存颜色指派的踪迹就可以了。

算法 8.1 -COLORREC (递归)

输入：无向图 $G=(V,E)$.

输出：G 的顶点的 3 着色 $c[1..n]$ ，其中每个 $c[j]$ 为 1,2 或 3。

1. **for** $k \leftarrow 1$ **to** n
2. $c[k] \leftarrow 0$
3. **end for**
4. $flag \leftarrow false$
5. $graphcolor(1)$
6. **if** $flag$ **then output** c
7. **else output** “no solution”

过程 $graphcolor(k)$

1. **for** $color=1$ **to** 3
2. $c[k] \leftarrow color$
3. **if** c 为合法着色 **then set** $flag \leftarrow true$ **and exit**
4. **else if** c 是部分的 **then** $graphcolor(k+1)$
5. **end for**

算法 8.2-COLORITER (迭代)

输入：无向图 $G=(V,E)$.

输出： G 的顶点的 3 着色 $c[1..n]$ ，其中每个 $c[i]$ 为 1,2 或 3。

```
1. for  $k \leftarrow 1$  to  $n$ 
2.      $c[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $k \leftarrow 1$ 
6. while  $k > 0$ 
7.     while  $c[k] \leq 2$ 
8.          $c[k] \leftarrow c[k] + 1$ 
9.         if  $c$  为合法着色, then 设置  $flag \leftarrow true$ 
            且从两个 while 循环退出
10.        else if  $c$  是部分解 then  $k \leftarrow k + 1$  { 前进 }
11.    end while
12.     $c[k] \leftarrow 0$ 
13.     $k \leftarrow k - 1$  { 回溯 }
14. end while
15. if  $flag$  then output  $c$ 
16. else output "no solution"
```

8.1.1 图着色——复杂度

- 对于这两种算法的时间复杂性，我们注意到在最坏情况下生成了 $O(3^n)$ 个节点。对于每个生成的节点，如果当前着色是合法的、部分的，或者二者都不是，这需要 $O(n)$ 的工作来检查。因此，在最坏情况下，全部的运行时间是 $O(n3^n)$ 。
- 空间复杂度是 $\Theta(n)$ 。

8.1.2 皇后问题——问题描述

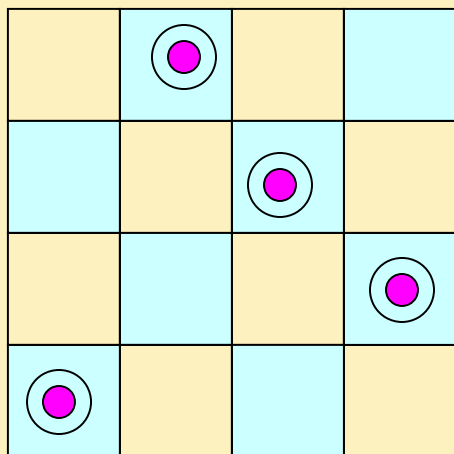
? 8 皇后问题描述

- 如何在 8×8 的国际象棋棋盘上安排 8 个皇后，使得没有两个皇后能互相攻击？
- 也就是说：如果两个皇后处在同一行、同一列或同一条对角线上，则她们能互相攻击。
- 为了简化讨论，我们将研究 4 皇后问题。

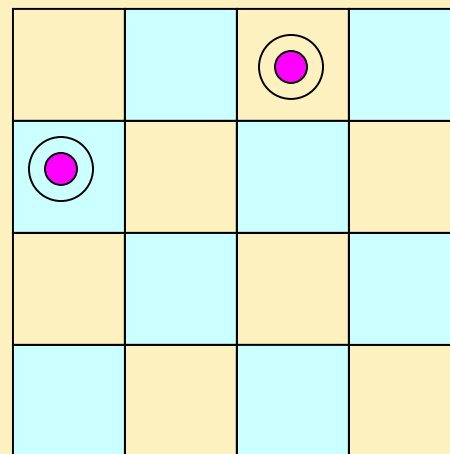
8.1.1 8 皇后问题——简化版 4 皇后

- 考虑一个 4×4 的棋盘，由于没有两个皇后能处在同一行，所以每个皇后都在不同的行上。又因为每行有 4 个位置，就有 4^4 种可能的布局，每种可能的布局可以用一个有 4 个分量的向量 $x = (x_1, x_2, x_3, x_4)$ 来描述。
- 实际上，因为没有两个皇后能放在同一列上，一个合法的放置对应于数 1, 2, 3, 4 的一个排列，这把搜索空间从 4^4 减少到 $4!$ 。

8.1.1 8 皇后问题——求解过程



The vector
(2,3,4,1)



The partial vector
(3,1)

8.1.1 8 皇后问题——互相攻击

假设：问题的解由 $(x_1, \dots, x_n)$ 表示

- 因为每行只放一个皇后，两个皇后不能处在同一行上。
- 如果两个皇后处在同一列上，他们互相攻击
 - In this case, $x_i = x_j$;
- 如果两个皇后处在同一条对角线上，他们互相攻击
 - In this case, $x_i - x_j = i - j$ or $x_i - x_j = j - i$.

算法 8.3 4-QUEENS(迭代)

输入：空。

输出：对应于 4 皇后问题的解的向量 $x[1..4]$ 。

```
1.  for  $k \leftarrow 1$  to 4
2.       $x[k] \leftarrow 0$  { 没有皇后放置在棋盘上 }
3.  end for
4.  flag  $\leftarrow$  false
5.   $k \leftarrow 1$ 
6.  while  $k \leq 4$ 
7.      while  $x[k] \leq 3$ 
8.           $x[k] \leftarrow x[k] + 1$ 
9.          if  $x$  为合法解 then set flag  $\leftarrow$  true 且从两个 while 循环退出
10.         else if  $x$  是部分解 then  $k \leftarrow k + 1$  { 前进 }
11.     end while
12.      $x[k] \leftarrow 0$ 
13.      $k \leftarrow k - 1$  { 回溯 }
14. end while
15. if flag then output  $x$ 
16. else output “no solution”
```

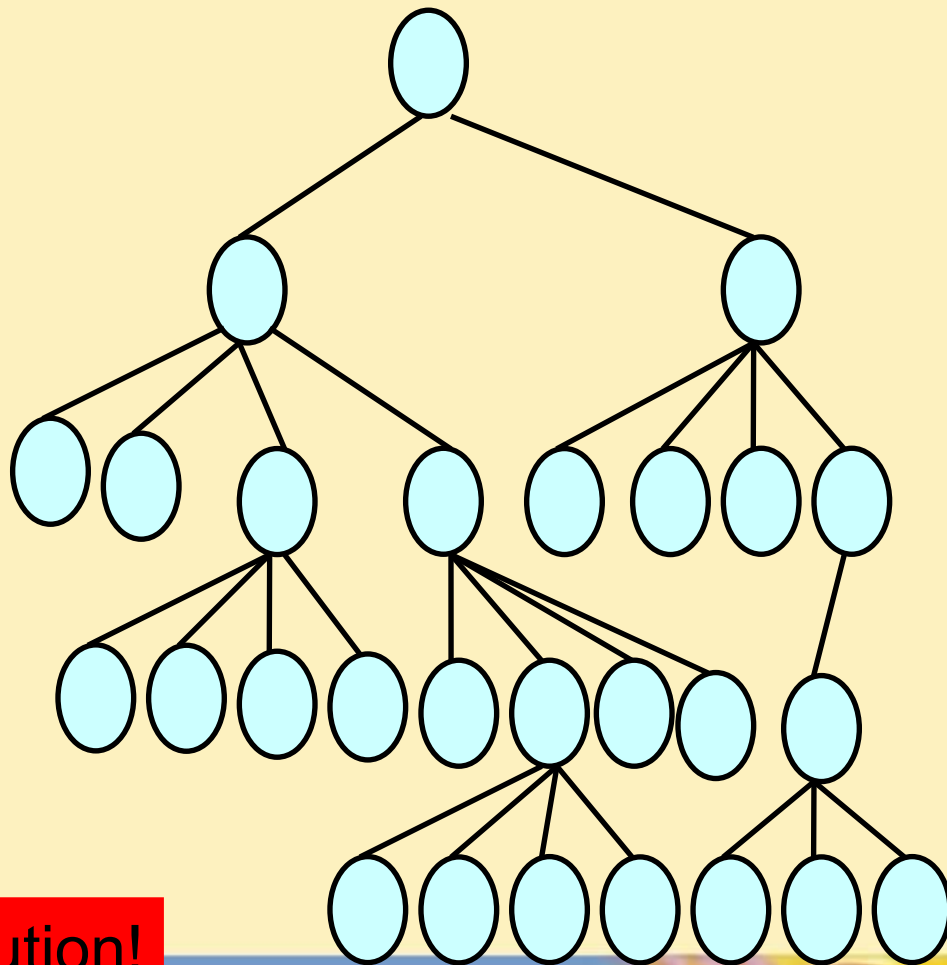
8.1.2 8 皇后——搜索树

1			
X	X	2	
X	X	X	X

1			
			2
X	3		
X	X	X	X

1			
			2
		X	X

	1		
X	X	X	2
3			
X	X	4	



Find a solution!

8.1.1 8 皇后——复杂度

- 现在考虑一个求解一般的 n 皇后问题的蛮力方法
 - 由于没有两个皇后能放在同一列上，则解向量必须是数 $1, 2, \dots, n$ 的一个排列。这样，蛮力方法可以改进为测试 $n!$ 种布局而不是 n^n 种。
 - 考虑 $(n-2)!$ 个向量，它们对应于前两个皇后放在第一列的那些布局，蛮力方法要盲目地测试所有这些向量，而在回溯法中用 $O(1)$ 次测试就可以避免这些测试。
 - 尽管回溯法在最坏情况下要用 $O(n^n)$ 时间来求解 n 皇后问题，根据经验它在有效性上远远超过蛮力方法的 $O(n!)$ 时间，作为它的可期望运行时间通常要快得多。

8.1.2 解长度的类型

- Representation of solution (解的表现形式)
 - 一般回溯算法，它可以作为一种系统的搜索方法应用到一类搜索问题当中，这类问题的解由满足事先定义好的某个约束的向量 $(x_1, x_2, \dots, x_i)$ 组成。这里 i 是 0 到 n 之间的某个整数，其中 n 是一个取决于问题规模的常量。在已经提到的两种算法——3 着色和 8 皇后问题中， i 是固定不变的。
 - 然而在一些问题中， i 可以像下面的例子展示的那样，对于不同的解可能有所不同。

8.1.2 解长度的类型

- 给定一个 n 个整数的集合 $X = \{x_1, x_2, \dots, x_n\}$ 和整数 y ，找出和等于 y 的 X 的子集 Y 。比如说，如果 $X = \{10, 20, 30, 40, 80, 60\}$ and $y = 60$
- 变长解：
 - 三种不同长度的解：
 $\{10, 20, 30\}$
 $\{20, 40\}$
 $\{60\}$
- 定长解：
 - 这个问题可以用另一种方法明确表达，解是一种明显的长度为 n 的布尔向量：
 $\{1, 1, 1, 0, 0, 0\}$
 $\{0, 1, 0, 1, 0, 0\}$
 $\{0, 0, 0, 0, 0, 1\}$.

回溯法的定义

- **回溯法**（backtracking）是一种系统地搜索问题解的方法。为实现回溯，首先需要定义一个**解空间**（solution space），然后以**易于搜索**的方式组织解空间，最后用**深度优先**的方法搜索解空间，获得问题的解。

回溯法的步骤

- 回溯法求解的三个步骤：
 - (1) 定义一个解空间，它包含问题的解
 - (2) 用易于搜索的方式组织解空间
 - (3) 深度优先搜索解空间，获得问题的解。

8.2 回溯策略的算法设计思想及分析方法

- 8.2.1 问题要求与约束函数、问题解的表示、状态空间树
- 8.2.2 回溯策略的基本思想
- 8.2.3 回溯算法一般框架
- 8.2.4 回溯策略的本质 - 约束穷举

8.2.1 回溯法的问题要求

- 问题要求：

- (1) 所要求的解必须能表示成一个 n -元组 $(x_1, x_2, \dots, x_n)$ ，其中 x_i 是取自某个有穷集 S_i 。通常，所求解的问题需要求取一个使某一个规范函数 $P(x_1, x_2, \dots, x_n)$ 取极值或满足规范条件的向量，有时还要找出满足 P 的所有向量。
- (2) 许多问题还要求所有的解满足一组综合的约束条件，这些条件分两种类型：**显式约束**和**隐式约束**。
 - **显式约束**限定每个 x 只从一个给定的集合上取值，满足显式约束的所有元组确定一个可能的**解空间**；
 - **隐式约束**规定解空间中那些实际上满足规范函数的元组，因此，描述了 x_i 必须彼此相关的情况。

问题举例

- 8-皇后问题：在一个 8×8 的棋盘上放 8 个皇后，且使得每两个之间都不能互相“攻击”，即都不出现在同一行、同一列及同一条斜对角线上。
 - 解的表示：假定第 i 个皇后放在第 i 行，用 8-元组 $(x_1, x_2, \dots, x_8)$ 表示解，其中 x_i 表示放置皇后 i 的列号。
 - 显示约束： $S_i = (1, 2, 3, \dots, 8), 1 \leq i \leq 8$
 - 隐含约束：没有两个 $x_i (1 \leq i \leq 8)$ 可以相同（在不同的列上），且没有两个皇后在同一斜对角线上。

8.2.1 问题解表示

(1) 解空间

设问题的解向量为 $X=(x_1, x_2, \dots, x_n)$ ， x_i 的取值范围为有穷集 S_i 。把 x_i 的所有可能取值组合，称为问题的解空间。每一个组合是问题的一个可能解。

例：0/1 背包问题， $S=\{0,1\}$ ，当 $n=3$ 时，0/1 背包问题的解空间是：

$\{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)\}$

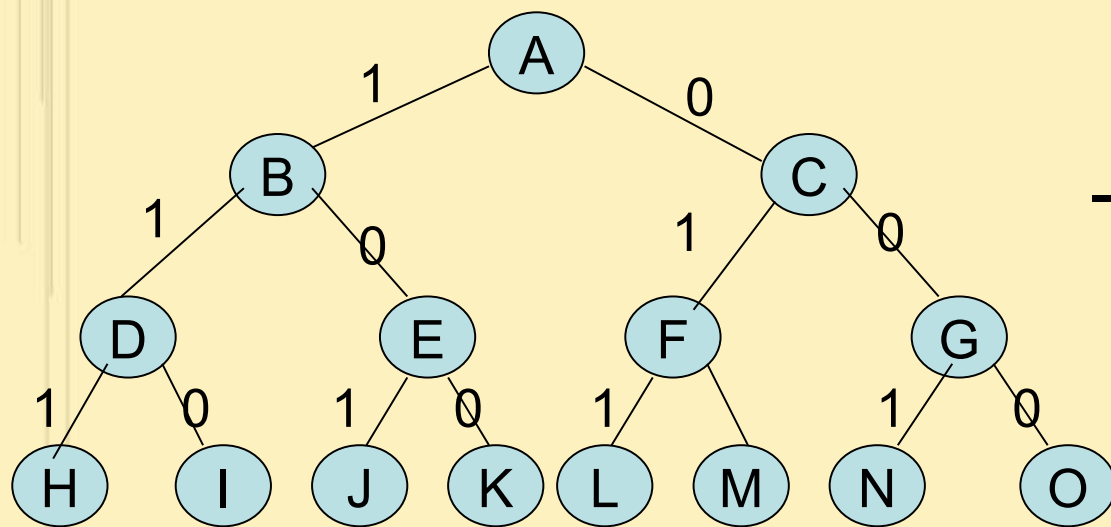
当输入规模为 n 时，有 2^n 种可能的解。

组合问题——问题包含元素所有可能的非空子集，看哪一个非空子集满足要求

8.2.1 回溯法的状态空间树

(2) 状态空间树：问题解空间的树形式表示

例： $n=3$, 0/1 背包问题的状态空间树。



---- 子集树

解空间大小： 2^n

8.2.1 回溯法的问题解表示

(1) 解空间

例：货郎担（旅行商）问题， $S=\{1,2,\dots,n\}$ ，当 $n=3$ 时， $S=\{1,2,3\}$ ，货郎担问题的解空间是：

$\{(1,1,1),(1,1,2),(1,1,3),(1,2,1),(1,2,2),(1,2,3),\dots,(3,3,1), (3,3,2),(3,3,3)\}$

当输入规模为 n 时，它有 n^n 种可能的解。

考虑到约束方程 $x_i \neq x_j$ 。因此，货郎担问题的解空间压缩为： $\{(1,2,3),(1,3,2),(2,1,3),(2,3,1),(3,1,2),(3,2,1)\}$

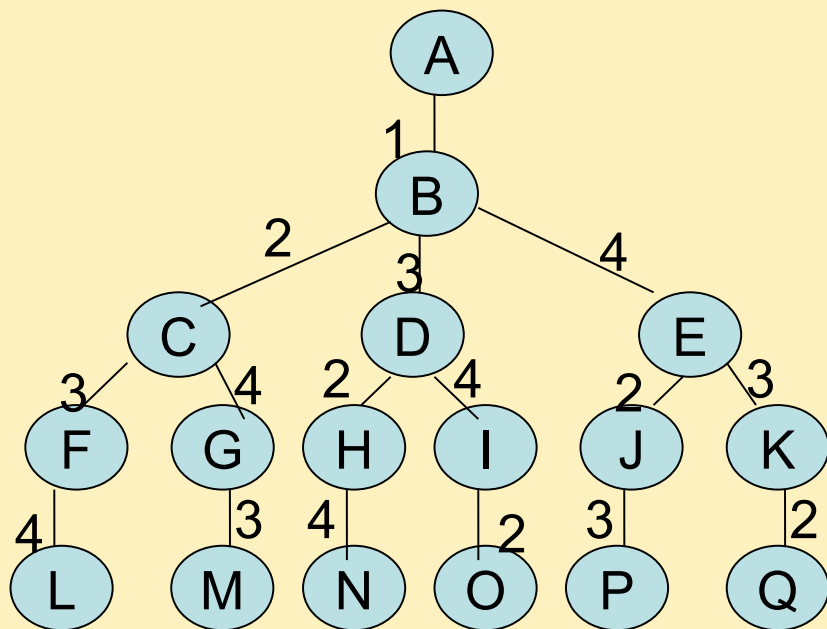
当输入规模为 n 时，它有 $n!$ 种可能的解。

排列问题——问题包含元素必须出现，看哪一个排列满足要求

8.2.1 回溯法的状态空间树

(2) 状态空间树：问题解空间的树形式表示

例： $n=4$ ，旅行商问题的状态空间树。



---- 排列树

解空间大小： $n!$

8.2.2 回溯基本思想

- 在回溯法中，**解向量**中每个 x_i 都属于一个有限的线序集 X_i ；因此，回溯算法按词典序考虑笛卡儿积 $X_1 \times X_2 \times \dots \times X_n$ 的元素
- 算法最初**从空向量开始**，然后选择 X_1 中最小元素作为 x_1 ，如果 (x_1) 是一个部分解，算法通过从 X_2 中选择最小的元素作为 x_2 继续，如果 (x_1, x_2) 是一个部分解，那么就包括 X_3 中最小的元素，否则 x_2 被置为 X_2 中的下一个元素。
- 一般地，假定算法已经检测到**部分解**为 $(x_1, x_2, \dots, x_j)$ ，它然后再去考虑向量 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 。我们有下面的情况：

8.2.2 回溯基本思想 (continue)

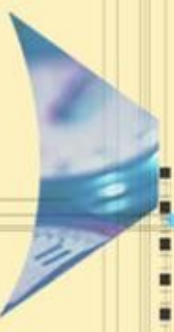
1. 如果 v 表示问题的最后解，算法记录下它作为一个解，在仅希望获得一个解时终止或者继续去找出其他解。
2. (向前步骤)。如果 v 表示一个部分解，算法通过选择集合 X_{j+2} 中的最小元素向前。
3. 如果 v 既不是最终的解，也不是部分解，则有两种子情况：
 - a) 如果从集合 X_{j+1} 中还有其他的元素可选择，算法将 x_{j+1} 置为 X_{j+1} 中的下一个
 - b) (回溯步骤)。如果从集合 X_{j+1} 中没有更多的元素可选择，算法通过将 x_j 置为 X_j 中的下一个元素回溯；如果从集合 X_j 中仍然没有其他的元素可以选择，算法通过将 x_{j-1} 置为 X_{j-1} 中的下一个元素回溯，依次类推。

8.2.2 回溯基本思想

- 对于任何一个问题，一旦设想出一种状态空间树，就可以系统地生成问题状态接着确定这些问题状态中的哪些状态是解状态，最后确定哪些解状态是答案状态，从而将问题解出。
- 问题状态的生成：从根结点开始生成其它结点。
- **活结点**：如果已生成一个结点而它的儿子结点还没有全部生成，则这个结点叫做活结点；当前正在生成其儿子结点的活结点叫 E- 结点 (**正在扩展的结点**)。
- **死结点**：不再进一步扩展或者其儿子结点已经全部生成的生成结点是死结点。

8.2.2 回溯基本思想

- 由根结点到其它结点的所有路径确定了这个问题的**状态空间**。解空间的树结构成为**状态空间树**。
- 树中的每个结点确定所求解问题的一个**问题状态**。
- **解状态**是这样一些问题状态 S ，对于这些问题状态，由根到 S 的那条路径确定了这解空间中的一个元组。
- **答案状态**是这样的一些解状态 S ，对于这些解状态而言，由根到 S 的这条路径确定了这问题的一个解。



8.2.3 回溯法的基本框架

- 非递归回溯框架
- 递归回溯框架



算法 8.5 BACKTRACKITER

输入：集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出：解向量 $v=(x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

1. $v \leftarrow ()$

2. $\text{flag} \leftarrow \text{false}$

3. $k \leftarrow 1$

4. **while** $k \geq 1$

5. **while** X_k 没有被穷举

6. $x_k \leftarrow X_k$ 中的下一个元素；将 x_k 加入 v

7. **if** v 为最终解 **then set** $\text{flag} \leftarrow \text{true}$

 且两个 **while** 循环退出

8. **else if** v 是部分解 **then** $k \leftarrow k+1$ { 前进 }

9. **end while**

10. 重置 X_k ，使得下一个元素排在第一位

11. $k \leftarrow k-1$

12. **end while**

13. **if** flag **then output** v

14. **else output** “no solution”

算法 8.4 BACKTRACKREC

输入：集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出：解向量 $v = (x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $\text{advance}(1)$
4. **if** flag **then output** v
5. **else output** “no solution”

过程 $\text{advance}(k)$

1. **for** each $x \in X_k$
2. $x_k \leftarrow x$; 将 x_k 加入 v
3. **if** v 为最终解 **then** set $\text{flag} \leftarrow \text{true}$ and **exit**
4. **else if** v 是部分解 **then** $\text{advance}(k+1)$
5. **end for**

8.2.4 回溯法的本质——约束穷举

- 问题的求解方法:

假定集合 S_i 的大小为 m_i ，于是就有 $m = m_1 m_2 \dots m_n$ 个 n -元组可能满足函数 P 。

- 问题的硬性处理求解方法:

构造出 m 个 n -元组并逐一测试它们是否满足 P ，从而找出该问题的所有最优解。

- 问题的回溯处理求解方法:

不断地用修改过的规范函数 $P_i(x_1, x_2, \dots, x_n)$ （有时称为限界函数）去测试正在构造中的 n -元组的部分向量 $(x_1, x_2, \dots, x_i)$ ，看其是否可能导致（最优）解。

如果判定 $(x_1, x_2, \dots, x_i)$ 不可能导致（最优）解，那么就将可能要测试的 $m_{i+1} m_{i+2} \dots m_n$ 个向量一概略去。

回溯法比硬性处理作的测试（ m 次）要少得多。

8.2.4 回溯法穷举——深度优先策略

- 1、确定了解空间的组织结构后，回溯法就从开始顶点（解空间树的根顶点）出发，以深度优先的方式搜索整个解空间。
- 2、这个开始顶点~~就~~成为一个活顶点，同时也成为当前的扩展顶点。在当前的扩展顶点处，搜索向纵深方向移至一个新顶点。这个新顶点就成为一个新的活顶点，并且成为当前的扩展顶点。
- 3、如果在当前的扩展顶点处不能再向纵深方向移动，则当前的扩展顶点就成为死顶点。此时应往回移动（回溯）至最近一个活顶点处，并使这个活顶点成为当前扩展顶点。
- 4、回溯法即以这种工作方式递归地在解空间中搜索，直至找到要求的解或解空间中已无活结点时为止。

8.2.4 回溯法约束——限界函数与剪枝函数

- 回溯法实际上在搜索的同时逐步生成解空间树（实际只需要一部分）。也就是说，对于实际问题我们不必要搜索整个解空间。
- 为了使搜索更加有效，我们常常在搜索过程中加一些判断以决定搜索是否该终止或改变路线。通常采用两种策略来避免无效的搜索，提高回溯法的搜索效率。其一是使用约束函数在扩展顶点处剪去不满足约束的子树；其二是用限界函数剪去不能得到最优解的子树。这两种函数统称为剪枝函数。

8.3. 回溯策略的经典应用（可选讲部分）

- 8.3.1 子集和问题的算法描述、算法分析、程序实现
- 8.3.2 哈密顿回路问题的算法描述、算法分析、程序实现
- 8.3.3 0/1 背包问题的算法描述、算法分析、程序实现
- 8.3.4 求解流水作业调度问题算法描述、算法分析、程序实现
- 8.3.5 货郎担问题问题的算法描述、算法分析、程序实现
- 8.3.6 限界函数与剪枝函数实践

8.3.1 子集和数问题

- 已知 $n+1$ 个正数: $w_i, (1 \leq i \leq n)$, 和 M , 要求找出 w_i 的和数是 M 的所有子集
- 解的表示形式:
 - 采用定长元组: 解为 n 元组 $(X_1, X_2, \dots, X_n)$, 其中 X_i 取 0 或 1, 表示 w_i 的子集合中是否出现。
- 约束条件: 并设定 $w_i (1 \leq i \leq n)$ 按非降次序排列。

$$\sum_{i=1}^n w(i)x(i) = M$$

限界函数

- $B_k(x(1), \dots, x(k)) = \text{true}$ 当且仅当：

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \leq M$$

- 假定 w_i 已经按照非降次序排列，则可以强化限界函数，如果下式成立， $B_k(x(1), \dots, x(k))$ 为假：

$$\sum_{i=1}^k w(i)x(i) + w(k+1) > M$$

- 故，假定 w_i 已经按照非降次序排列，限界函数 $B_k(x(1), \dots, x(k)) = \text{true}$ 当且仅当：

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \leq M \text{ 并且 } \sum_{i=1}^k w(i)x(i) + w(k+1) \leq M$$

算法 SUMOFSUBREC

输入 : $w[1,2,\dots,n]$ and M , $w[i]>0$, $M > 0$.

输出 : A solution vector $x[1,2,\dots,n]$, where each $x[i]$ is 0 or 1.

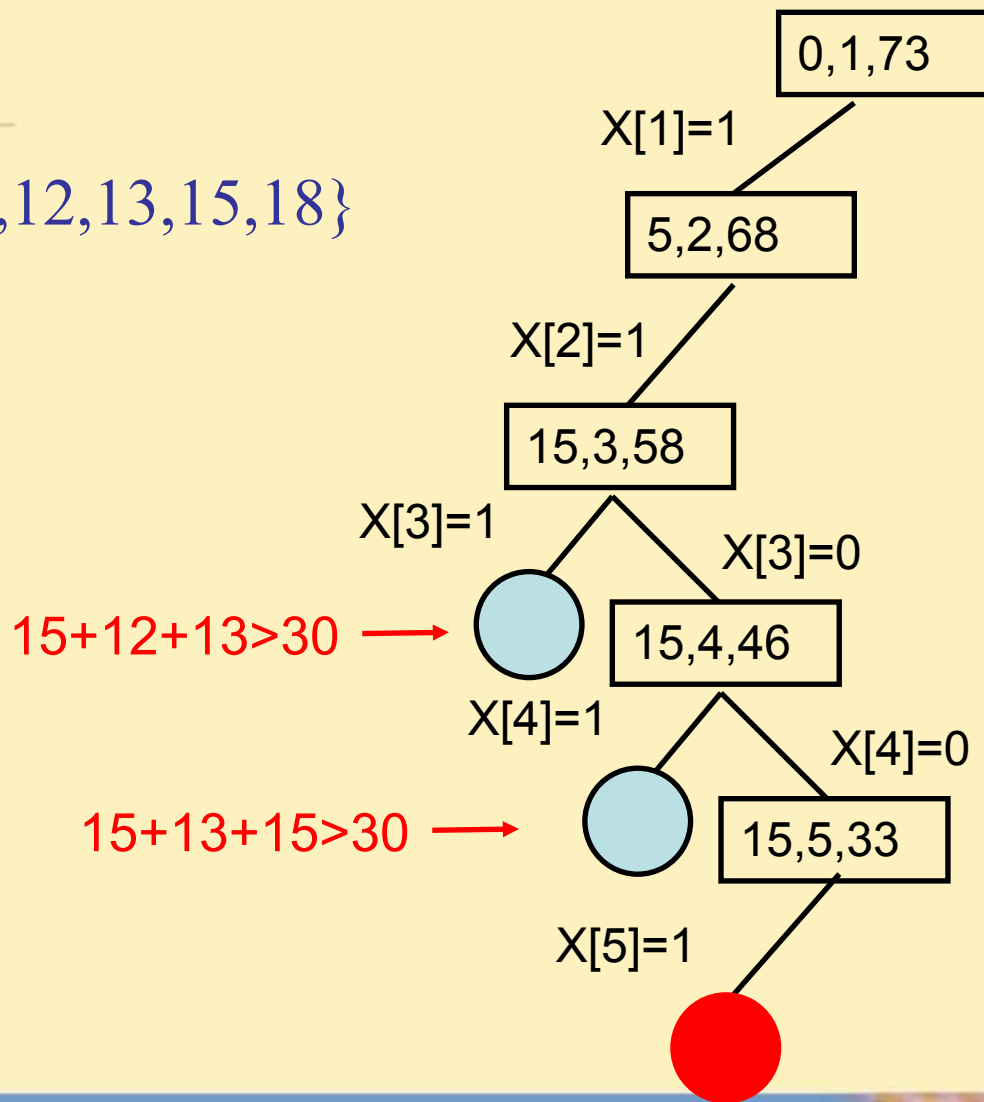
1. **for** $k \leftarrow 1$ **to** n
2. $x[k] \leftarrow 0$
3. **end for**
4. $flag \leftarrow false$
5. $s \leftarrow 0$; $r \leftarrow w[1]+w[2]+\dots+w[n]$
6. **if** $r \leq M$ **then** SUMOFSUB($s,1,r$) **endif**
7. **if** $flag$ **then output** x
8. **else output** "no solution"
9. **end if**

Procedure SUMOFSUB (s,k,r)

1. $x[k] \leftarrow 1$
2. **if** $s+w[k]=M$ // x 是合法解
3. **then set** $flag \leftarrow true$ **and exit**
4. **else**
5. **if** $s+w[k]+w[k+1] \leq M$ // x 是部分的 $B_k=true$, 第 k 个数选入
6. **then** SUMOFSUB($s+w[k],k+1,r-w[k]$) // 递归调用第 $k+1$ 个数
7. **end if**
8. **end if**
9. **if** $s+r-w[k] \leq M$ and $s+w[k+1] \leq M$ // $B_k=false$ 第 k 个数不选入
10. **then** $x[k] \leftarrow 0$, SUMOFSUB($s,k+1,r-w[k]$)
11. **end if**

Example

- $w[1..6] = \{5, 10, 12, 13, 15, 18\}$
 $M = 30$



8.3.2 哈密顿回路问题

假定图 $G=(V, E)$ 的顶点集为 $V=\{1, 2, \dots, n\}$ ，则哈密顿回路的可能解表示为 n 元组 $X=(x_1, x_2, \dots, x_n)$ ，其中， $x_i \in \{1, 2, \dots, n\}$ ，并有如下约束条件：

$$\left\{ \begin{array}{l} (x_i, x_{i+1}) \in E \quad (1 \leq i \leq n-1) \\ (x_n, x_1) \in E \\ x_i \neq x_j \quad (1 \leq i, j \leq n, \quad i \neq j) \end{array} \right.$$

https://blog.csdn.net/nice___amusing

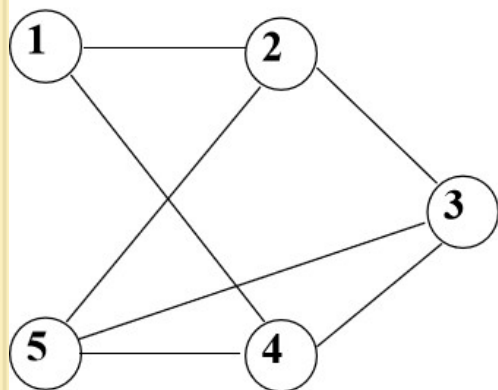
8.3.2 哈密顿回路问题

- 条件约束:

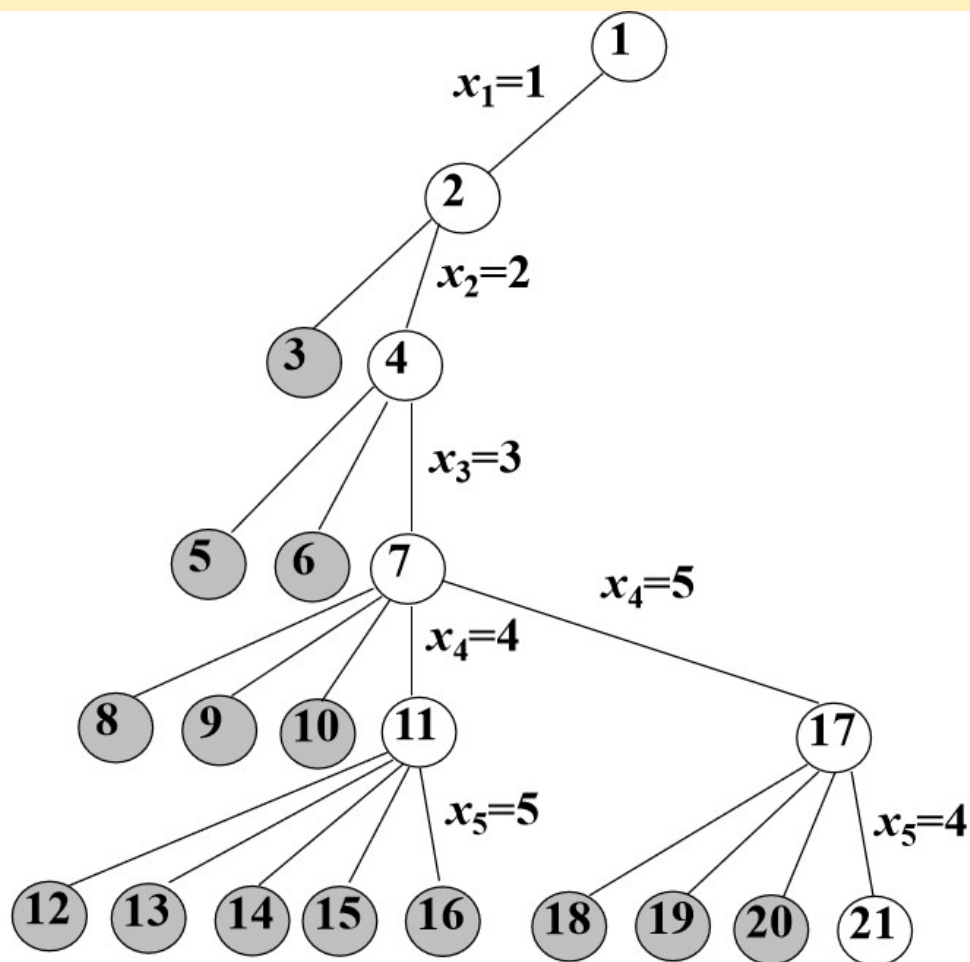
1. 加入路径的节点与前一个加入路径的顶点一定是连通的;
2. 加入路径的最后一个顶点和第一个顶点一定是连通的;
3. 路径上不能出现重复的顶点。



8.3.2 哈密顿回路问题



(a) 一个无向图



(b) 哈密顿回路的搜索空间

8.3.2 哈密顿回路问题

算法8.3——哈密顿回路问题

1. 将顶点数组 $x[n]$ 初始化为0, 标志数组 $visited[n]$ 初始化为0;
2. $k=1$; $visited[1]=1$; $x[1]=1$; 从顶点1出发构造哈密顿回路;
3. **while** ($k \geq 1$)
 - 3.1 $x[k]=x[k]+1$, 搜索下一个顶点;
 - 3.2 若(n 个顶点没有被穷举) 执行下列操作
 - 3.2.1 若(顶点 $x[k]$ 不在哈密顿回路上 $\&\&(x[k-1], x[k]) \in E$), 转步骤3.3;
 - 3.2.2 否则, $x[k]=x[k]+1$, 搜索下一个顶点;
 - 3.3 若数组 $x[n]$ 已形成哈密顿路径, 则输出数组 $x[n]$, 算法结束;
 - 3.4 否则,
 - 3.4.1 若数组 $x[n]$ 构成哈密顿路径的部分解, 则 $k=k+1$, 转步骤3;
 - 3.4.2 否则, 重置 $x[k]$, $k=k-1$, 取消顶点 $x[k]$ 的访问标志, 转步骤3;

https://blog.csdn.net/nice_amusin

8.3.3 0/1 背包问题

- 有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包。
- 设计从这些物品中选取一部分物品放入该背包的方案，每个物品要么选中要么不选中，要求选中的物品不仅能够放到背包中，而且满足重量限制具有最大的价值。

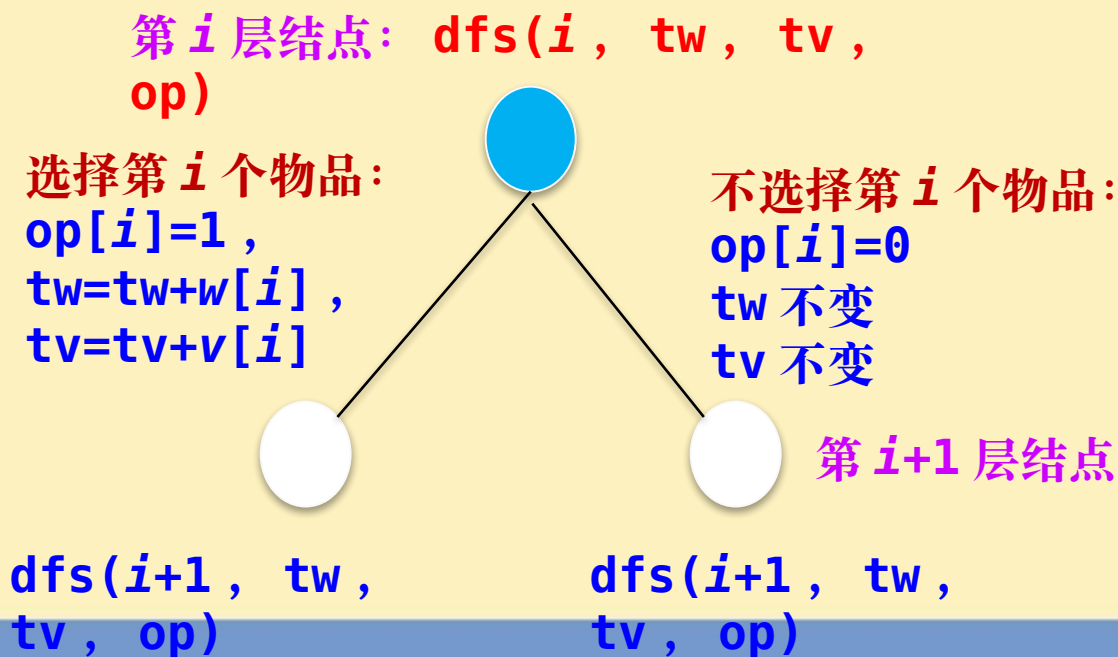
8.3.3 0/1 背包问题

1. 装入背包中物品重量和恰好为 W

- 用 $w[1..n]/v[1..n]$ 存放物品信息, $x[1..n]$ 数组存放最优解, 其中每个元素取 1 或 0, $x[i]=1$ 表示第 i 个物品放入背包中, $x[i]=0$ 表示第 i 个物品不放入背包中。
- 这是一个求最优解问题。找到更优解 $(op, tv) \Rightarrow (x, maxv)$ 。

8.3.3 0/1 背包问题

- 解空间是一棵子集树!
- 对第 i 层上的某个分枝结点, 对应的状态为 $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$, 其中 tw 表示装入背包中的物品总重量, tv 表示背包中物品总价值, op 记录一个解向量。该状态的两种扩展如下:

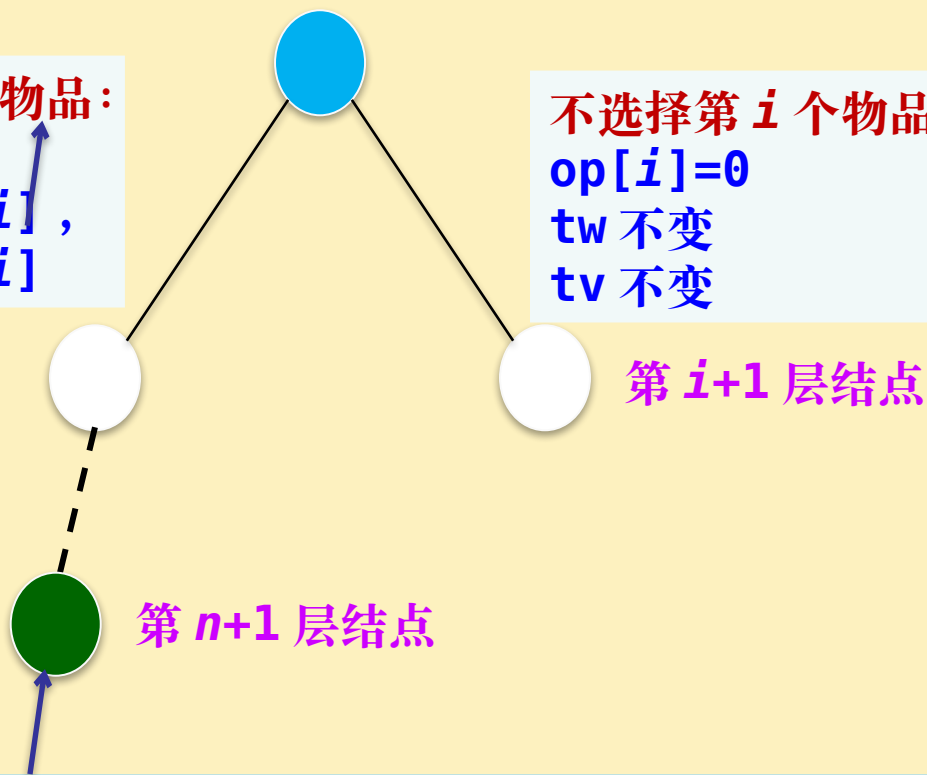


8.3.3 0/1 背包问题

第 i 层结点: $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$

选择第 i 个物品:
 $\text{op}[i]=1$,
 $\text{tw}=\text{tw}+w[i]$,
 $\text{tv}=\text{tv}+v[i]$

不选择第 i 个物品:
 $\text{op}[i]=0$
 tw 不变
 tv 不变



- 叶子结点表示已经对 n 个物品做了决策。
- 对所有叶子结点进行比较求出满足 $\text{tw}==W$ 的最大 maxv ，对应的最优解 op 存放到 x 中。

8.3.3 0/1 背包问题

0/1 背包问题 ($W=6$) :

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

(总重量 tw , 总价值)

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

1	5	4
2	3	4
3	2	3
4	1	1

0,0

根结点:
 $i=1$

1

0

x_1 : 选或不选物品 1

5,4

0,0

$i=2$

1

0

1

0

x_2 : 选或不选物品 2

$i=3$

8,8

5,4

3,4

0,0

x_3 : 选或不选物品 3

$i=4$

1

0

1

0

1

0

1

0

x_4 : 选或不选物品 4

$i=5$

10,1
1

8,8

7,7

5,4

5,7

3,4

2,3

0,0

11,1
2

10,1
1

9,9

8,8

8,8

7,7

6,5

5,4

6,8

5,7

4,5

3,4

3,4

2,3

1,1

0,0

最优解

8.3.3 0/1 背包问题

// 问题表示

int n=4;

//4 种物品

int W=6;

// 限制重量为 6

int w[]={0,5,3,2,1};

// 存放 4 个物品重量，不用下

标 0 元素

int v[]={0,4,4,3,1};

// 存放 4 个物品价值，不用下

标 0 元素

// 求解结果表示

int x[MAXN];

// 存放最终解

int maxv;

// 存放最优解的总价值

8.3.3 0/1 背包问题

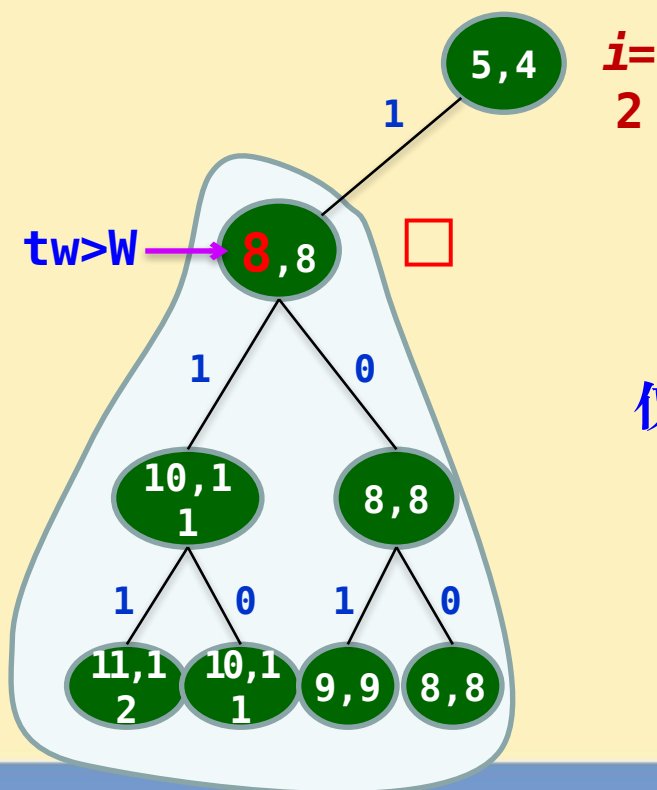
采用解空间为子集树递归的算法框架

```
void dfs(int i,int tw,int tv,int op[])
{  if (i>n)                                // 找到一个叶子结点
    {  if (tw==W && tv>maxv)                // 找到一个满足条件的更优解, 保存
        {  maxv=tv;
            for (int j=1;j<=n;j++)
                x[j]=op[j];
        }
    }

    else                                    // 尚未找完所有物品
    {  op[i]=1;                             // 选取第 i 个物品
        dfs(i+1,tw+w[i],tv+v[i],op);

        op[i]=0;                           // 不选取第 i 个物品, 回溯
        dfs(i+1,tw,tv,op);
    }
}
```

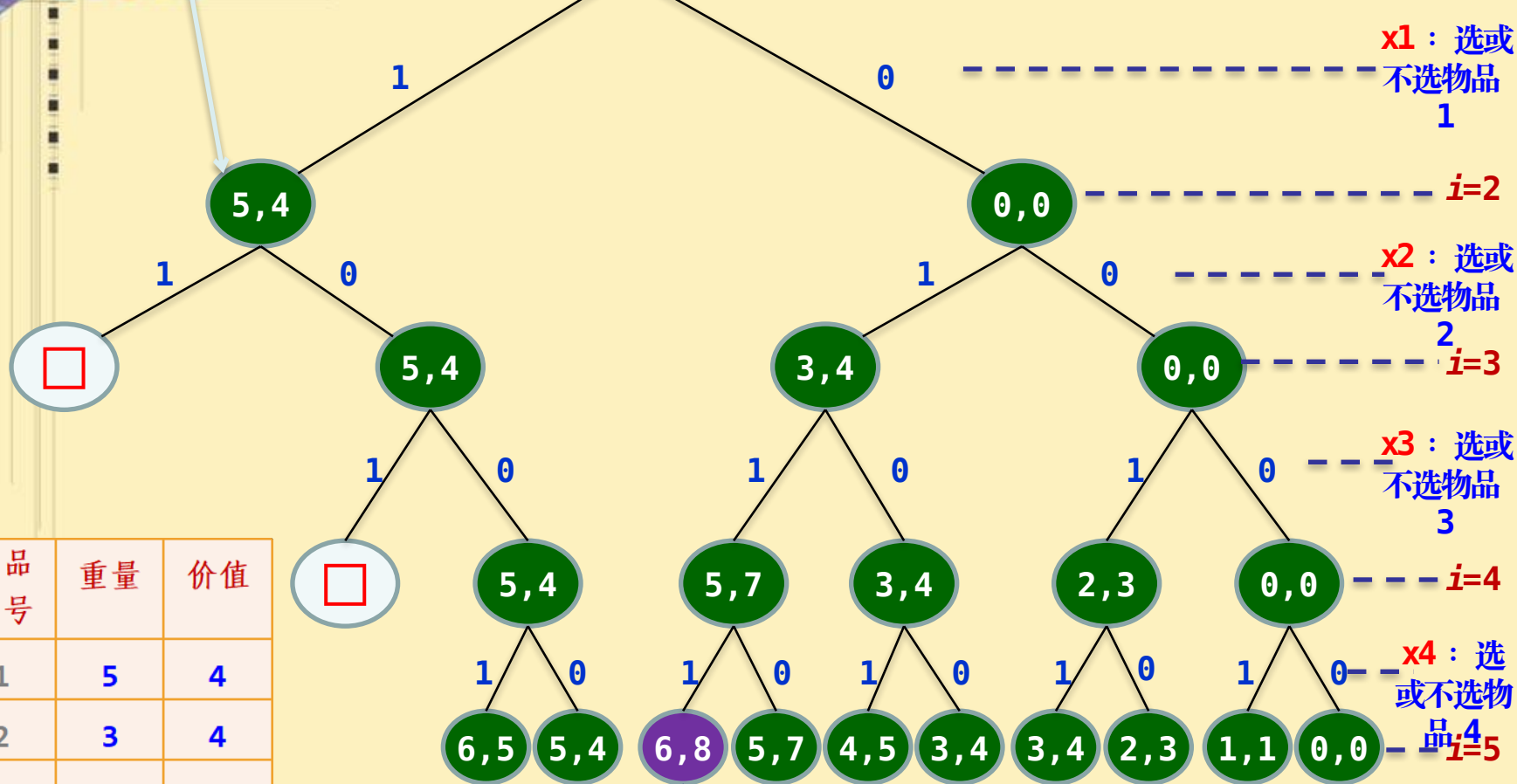

改进 1：左剪枝： 对于第 i 层的有些结点， $tw+w[i]$ 已超过了 W ($W=6$)，显然再选择 $w[i]$ 是不合适的。如第 2 层的 (5, 4) 结点 $\square$ 进行扩展是不必要的。



仅仅扩展满足 $tw+w[i] \leq W$ 的左孩子结点

(总重量 tw , 总价值 tv)

根结点:
 $i=1$



最优解

物品编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1

```

void dfs(int i,int tw,int tv,int op[])
{
    if (i>n)                                // 找到一个叶子结点
    {
        if (tw==W && tv>maxv)                // 找到一个满足条件的更优解，保存
        {
            maxv=tv;
            maxw=tw;
            for (int j=1;j<=n;j++)
                x[j]=op[j];
        }
    }
    else                                     // 尚未找完所有物品
    {
        if ( tw+w[i]<=W )                    // 左孩子结点剪枝
        {
            op[i]=1;                         // 选取第 i 个物品
            dfs(i+1,tw+w[i],tv+v[i],op);
        }
        op[i]=0;                             // 不选取第 i 个物品，回溯
        dfs(i+1,tw,tv,op);
    }
}

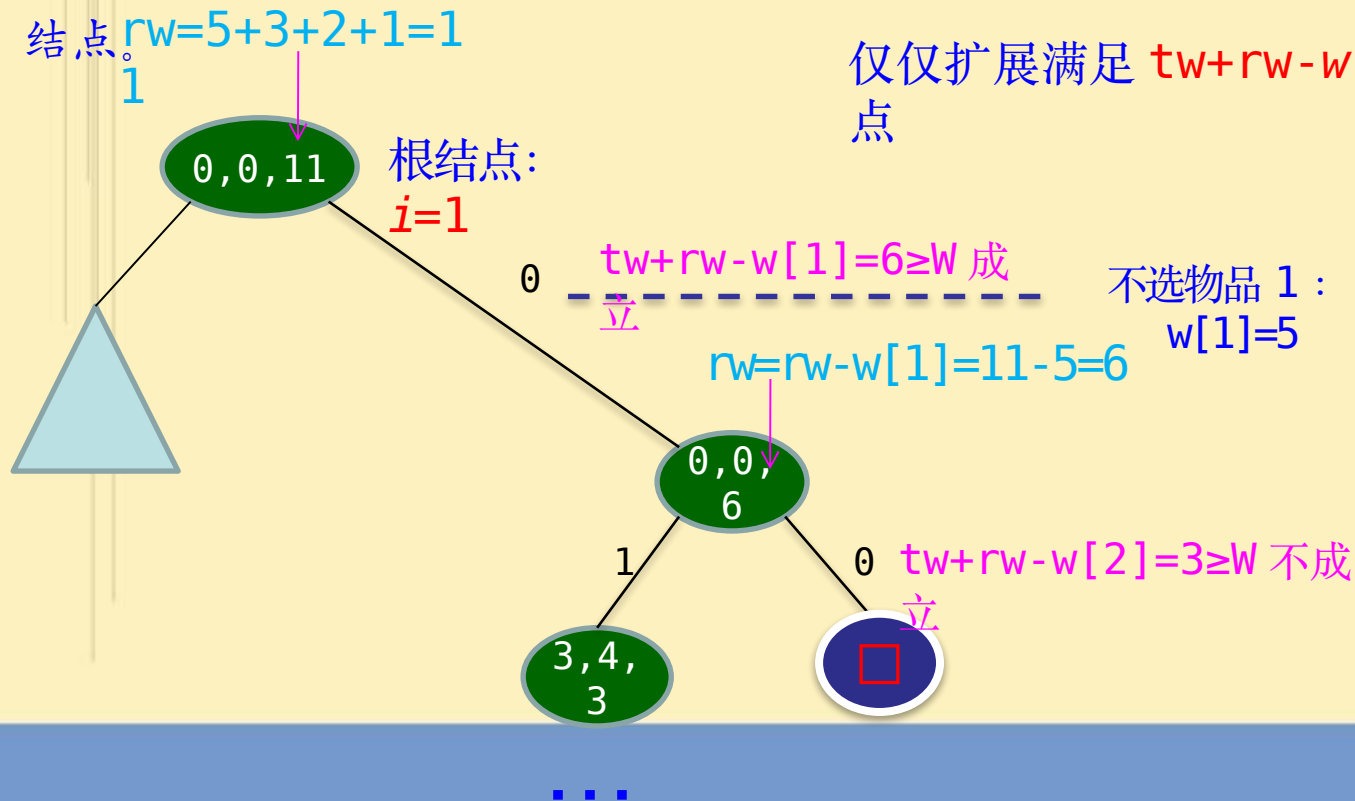
```

左剪枝



改进 2：右剪枝： $rw=w[i]+w[i+1]+\dots+w[n]$ 。

当不选择物品 i 时： $rw-w[i]=w[i+1]+\dots+w[n]$ ，若 $tw+rw-w[i]<W$ ，也就是说即使选择后面的所有物品，重量也不会达到 W ，因此不必要再考虑扩展这样的



```

void dfs(int i,int tw,int tv,int rw,int op[])
{ // 初始调用时 rw 为所有物品重量和
  int j;
  if (i>n) // 找到一个叶子结点
  { if (tw==W && tv>maxv) // 找到一个满足条件的更优解, 保存
    { maxv=tv;
      for (j=1;j<=n;j++) // 复制最优解
        x[j]=op[j];
    }
  }
  else // 尚未找完所有物品
  { if (tw+w[i]<=W) // 左孩子结点剪枝
    { op[i]=1; // 选取第 i 个物品
      dfs(i+1,tw+w[i],tv+v[i],rw-w[i],op);
    }
    if ( tw+rw-w[i]>=W )
    { op[i]=0; // 不选取第 i 个物品, 回溯
      dfs(i+1,tw,tv,rw-w[i],op);
    }
  }
}
}

```





【算法分析】 该算法不考虑剪枝时解空间树中有 $2^{n+1}-1$ 个结点（剪枝的结点个数不确定），所以最坏情况下算法的时间复杂度为 $O(2^n)$ 。



2. 装入背包中物品重量和不超过

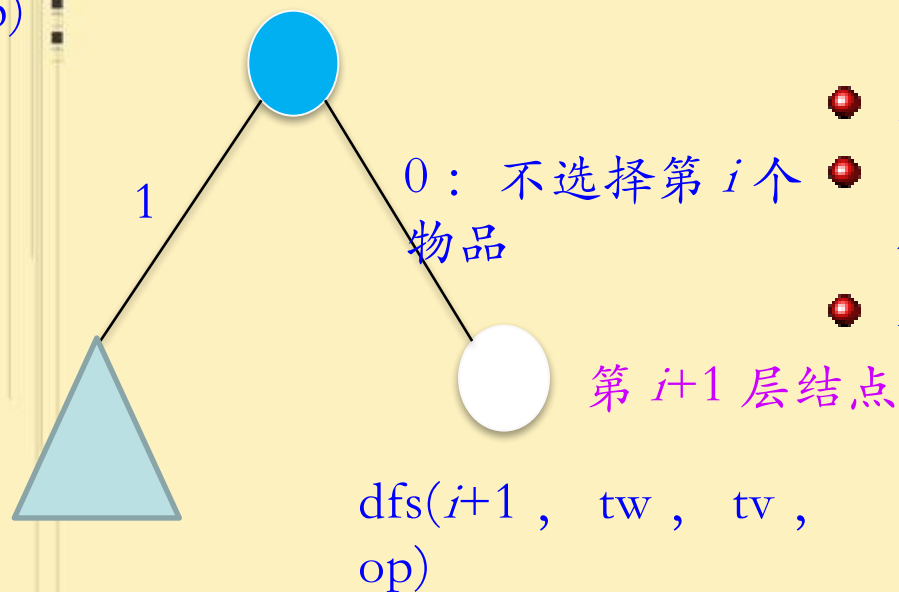
W

问题变为求背包中物品重量和不超过 W 的最大价值的装入方案：

- 前面左剪枝方式不变。
- 但右剪枝方式不再有效，改为采用上界函数进行右剪枝。

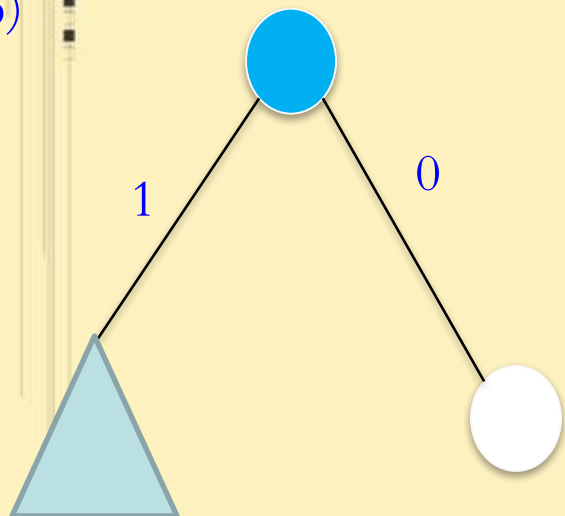


第 i 层结点: $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$



- 上界函数 $\text{bound}(i) = \text{tv} + r$
- 表示沿着该方向选择得到物品的价值上界
- r 表示剩余物品的总价值

第 i 层结点: $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$



- 上界函数 $\text{bound}(i) = \text{tv} + r$
- 假设当前求出最大价值 maxv ，若 $\text{bound}(i) \leq \text{maxv}$ ，则右剪枝，否则继续扩展。
- 显然 r 越小， $\text{bound}(i)$ 也越小，剪枝越多，为了构造更小的 r ，将所有物品以单位重量价值递减排列。

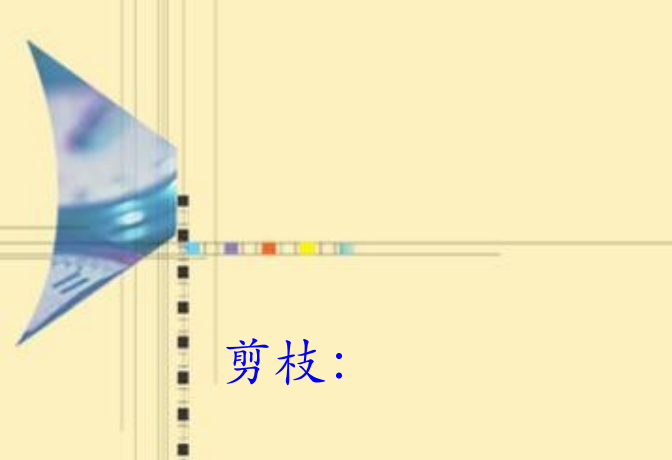
第 i 层结点: $\text{dfs}(i, \text{tw}, \text{tv}, \text{op})$

0: 不选择第 i 个物品

第 $i+1$ 层结点

叶子结点

```
int bound(int i,int tw,int tv)    // 求上界
{  i++;                          // 从 i+1 开始
  while (i<=n && tw+A[i].w<=W)    // 若序号为 i 的物品可以整个放入
  {  tw+=A[i].w;
    tv+=A[i].v;
    i++;
  }
  if (i<=n)
    return tv+(W-tw)*A[i].p;      // 序号为 i 的物品不能整个放入
  else
    return tv;
}
```



剪枝:

- 左剪枝: 仅仅扩展 $tw + w[i] \leq W$ 的左孩子结点
 - 右剪枝: 仅仅扩展 $bound(i, tw, tv) > maxv$ 的右孩子结点
- 点



一旦找到一个解后, 后面找到的其他解 (tv , op) 只能越来越优



```

void dfs(int i,int tw,int tv,int op[]) // 求解 0/1 背包问题
{
    if (i>n) // 找到一个叶子结点
    {
        maxv=tv; // 存放更优解
        for (int j=1;j<=n;j++)
            x[j]=op[j];
    }
    else // 尚未找完所有物品
    {
        if (tw+A[i].w<=W) // 左孩子结点剪枝
        {
            op[i]=1; // 选取序号为 i 的物品
            dfs(i+1,tw+A[i].w,tv+A[i].v,op);
        }
        if (bound(i,tw,tv)>maxv) // 右孩子结点剪枝
        {
            op[i]=0; // 不选取序号为 i 的物品，回溯
            dfs(i+1,tw,tv,op);
        }
    }
}

```

0/1 背包问题 ($W=6$) :

物品编号	重量	价值	$p=v/w$
1	5	4	0.8
2	3	4	1.3
3	2	3	1.5
4	1	1	1



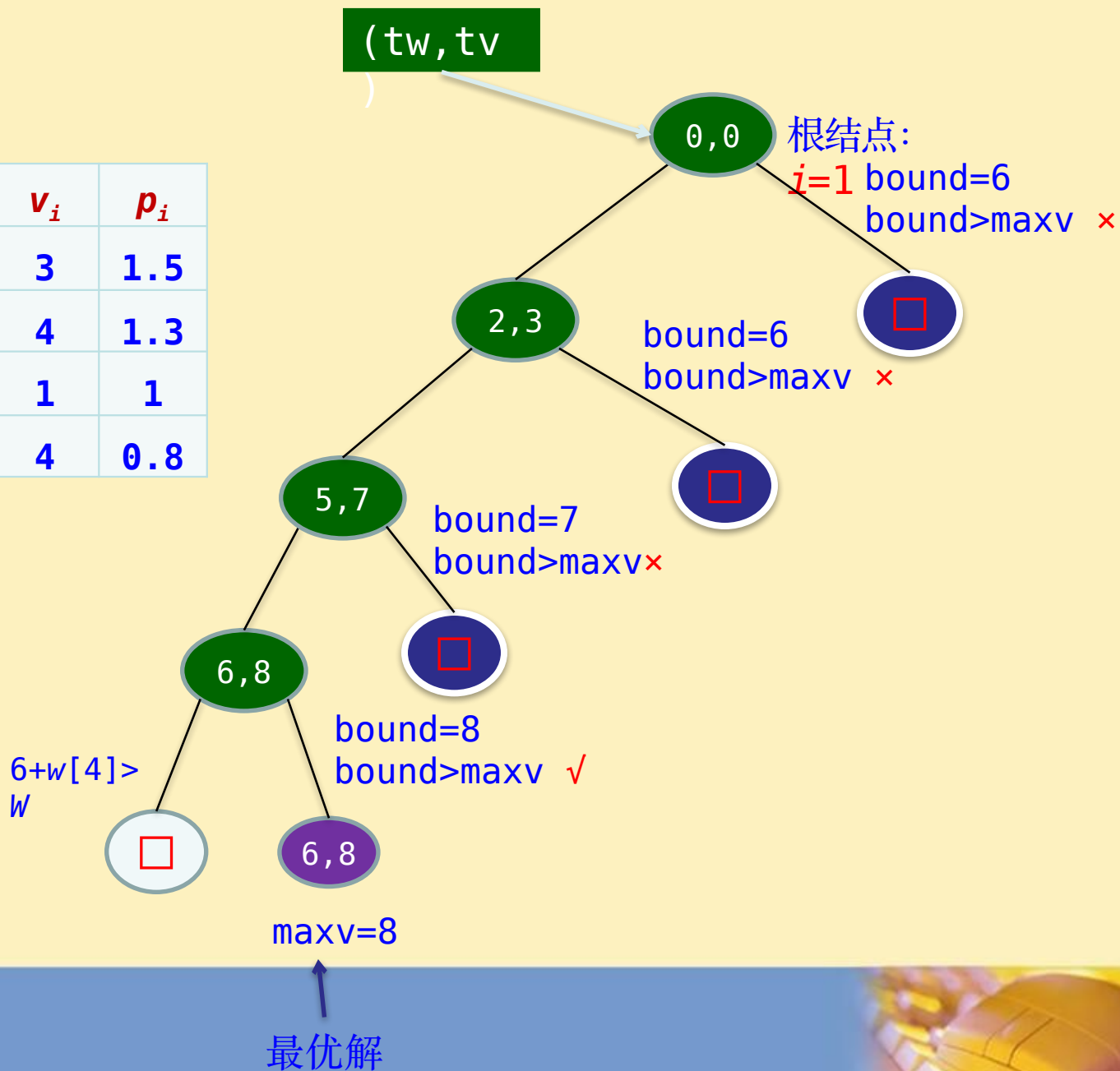
A 数组 :

i	no	w_i	v_i	p_i
1	3	2	3	1.5
2	2	3	4	1.3
3	4	1	1	1
4	1	5	4	0.8



$W=6, \max v=0$

i	no	w_i	v_i	p_i
1	3	2	3	1.5
2	2	3	4	1.3
3	4	1	1	1
4	1	5	4	0.8





【算法分析】该算法不考虑剪枝时解空间树中有 $2^{n+1}-1$ 个结点（剪枝的结点个数不确定），所以最坏情况下算法的时间复杂度为 $O(2^n)$ 。

8.3.4 求解流水作业调度问题

【问题描述】 有 n 个作业（编号为 $1 \sim n$ ）要在由两台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工，然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i 和 b_i ($1 \leq i \leq n$)。

流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 M_1 上开始加工，到最后最后一个作业在机器 M_2 上加工完成所需的时间最少。可以假定任何作业一旦开始加工，就不允许被中断，直到该作业被完成，即非优先调度。

例子

【输入格式】输入包含若干个用例。每个用例第一行是作业数 n ($1 \leq n \leq 1000$)，接下来 n 行，每行两个非负整数，第 i 行的两个整数分别表示在第 i 个作业在第一台机器和第二台机器上加工时间。以输入 $n=0$ 结束。

【输出格式】每个用例输出一行，表示采用最优调度所用的总时间，即从第一台机器开始到第二台机器结束的时间。

【输入样例】

4

5 6

12 2

4 14

8 7

0

【输出样例】

33

作业编号	1	2	3	4
M_1 时间 a	5	12	4	8
M_2 时间 b	6	2	14	7

问题解空间

- 所有的作业都必须做完
- 采用回溯法求解，对应的解空间是一个是排列树，相当于求出 n 个作业的一种排列使完成时间最少。

n 个作业的全排列



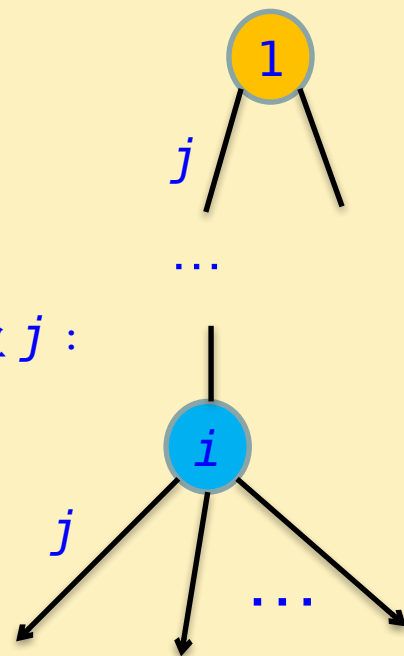
完成时间最少

n 个作业的某个排列：最优解

$x[1]$

设置第 i 步执行的作业 j ：
 $x[i]=j$

$x[i]$



排列的解

- f1 数组: $f1[i]$ 表示第 i 步执行的作业 $x[i]$ 在 M1 上执行完的总时间 (含前面作业的执行时间)
- f2 数组: $f2[i]$ 表示第 i 步执行的作业 $x[i]$ 在 M2 上执行完的总时间 (含前面作业的执行时间)
- 由于一个作业总是先在 M1 上执行后在 M2 执行, 所以 $f2[n]$ 就是执行全部作业的总时间。

$x = \{ x[1], x[2], \dots, x[i], \dots, x[n] \}$

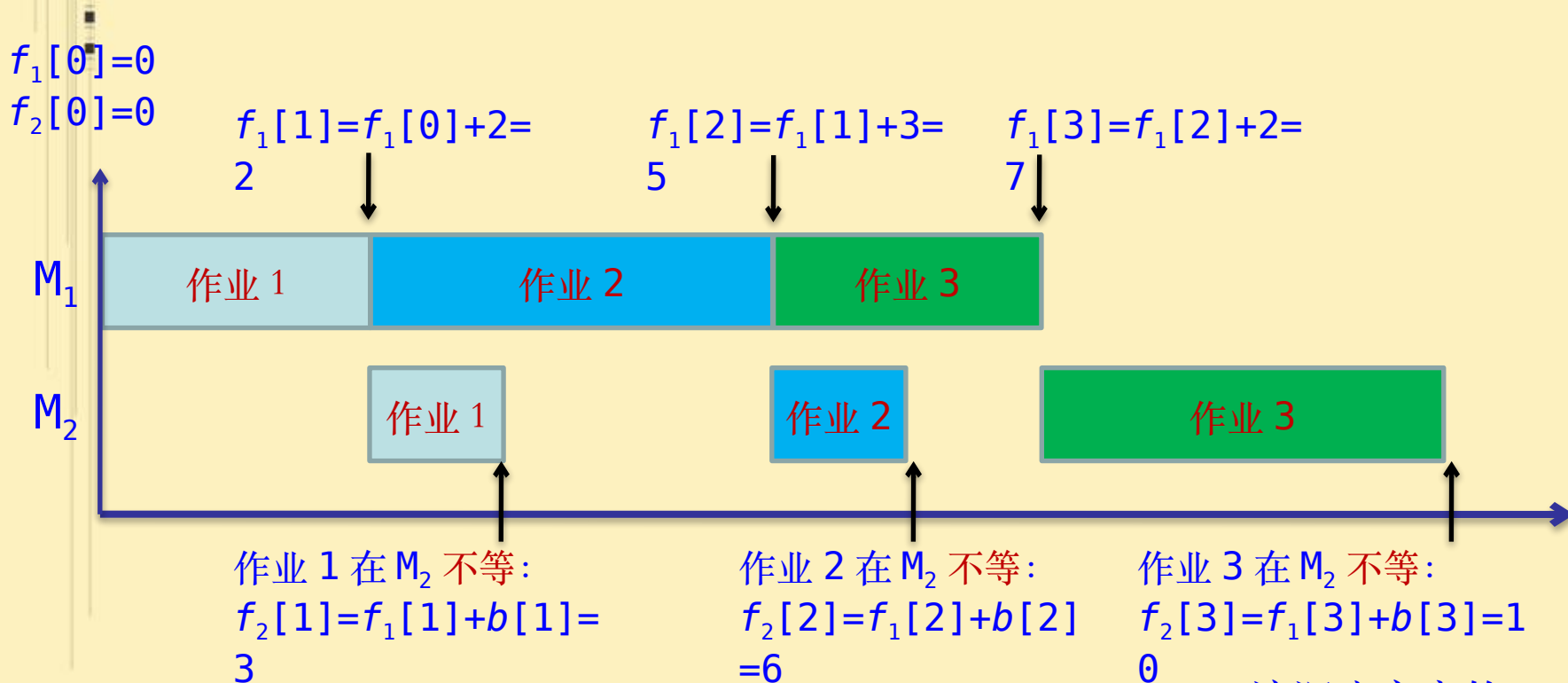


第 i 步执行的作业

一个示例，假设有 3 个作业：

作业编号	1	2	3
M_1 时间 a	2	3	2
M_2 时间 b	1	1	3

现在的调用方案为 $x = (1, 2, 3)$ ，即按作业 1、2、3 的顺序执行。首先将 f_1 和 f_2 数组所有元素初始化为 0。该调度方案的总时间计算如下：

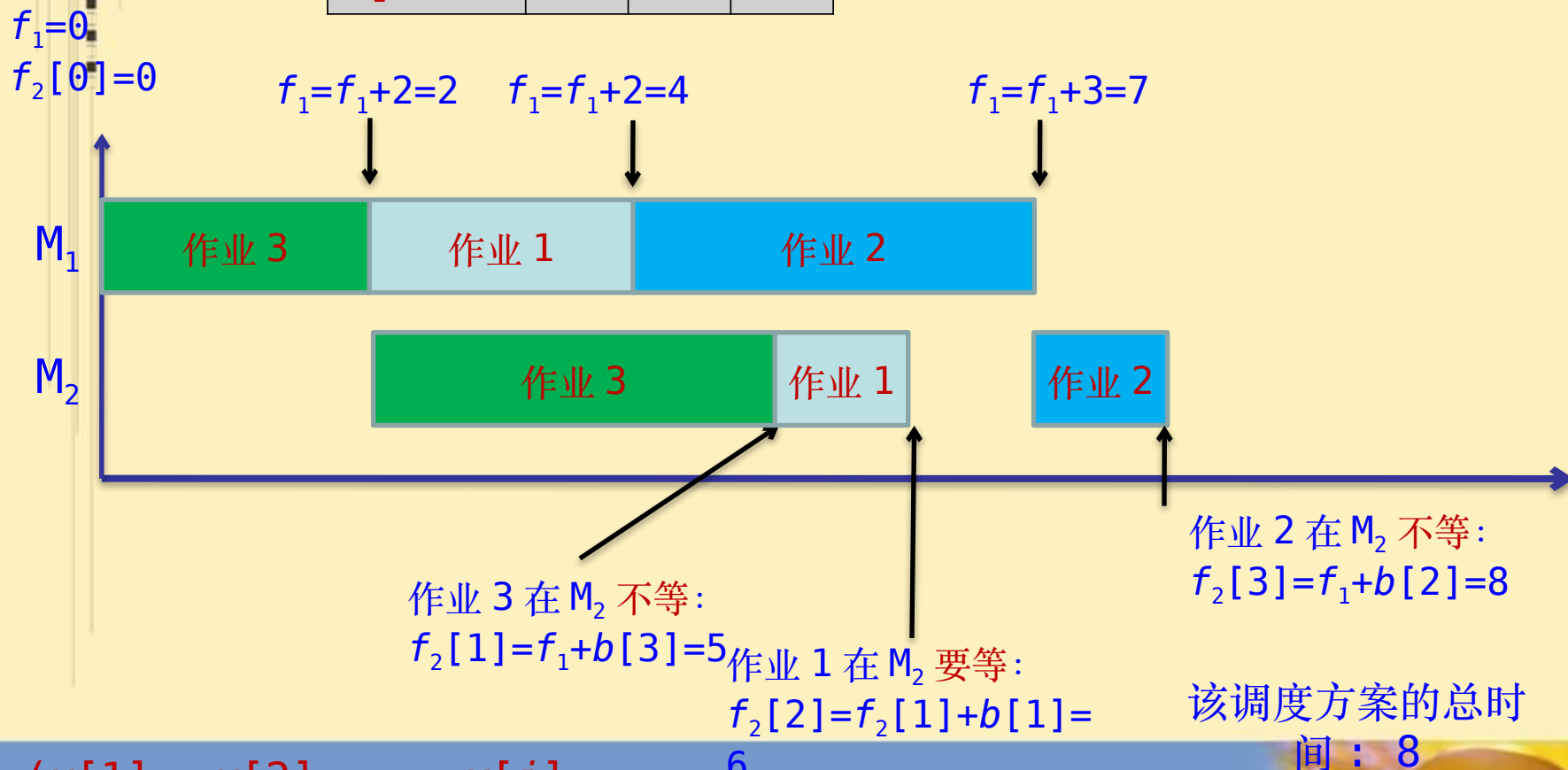


由于每个作业都是从 M_1 开始的，即 M_1 上各个作业是连续执行的，不需要等待，所以 f_1 不需要用数组表示，直接用单个变量 f_1 表示。

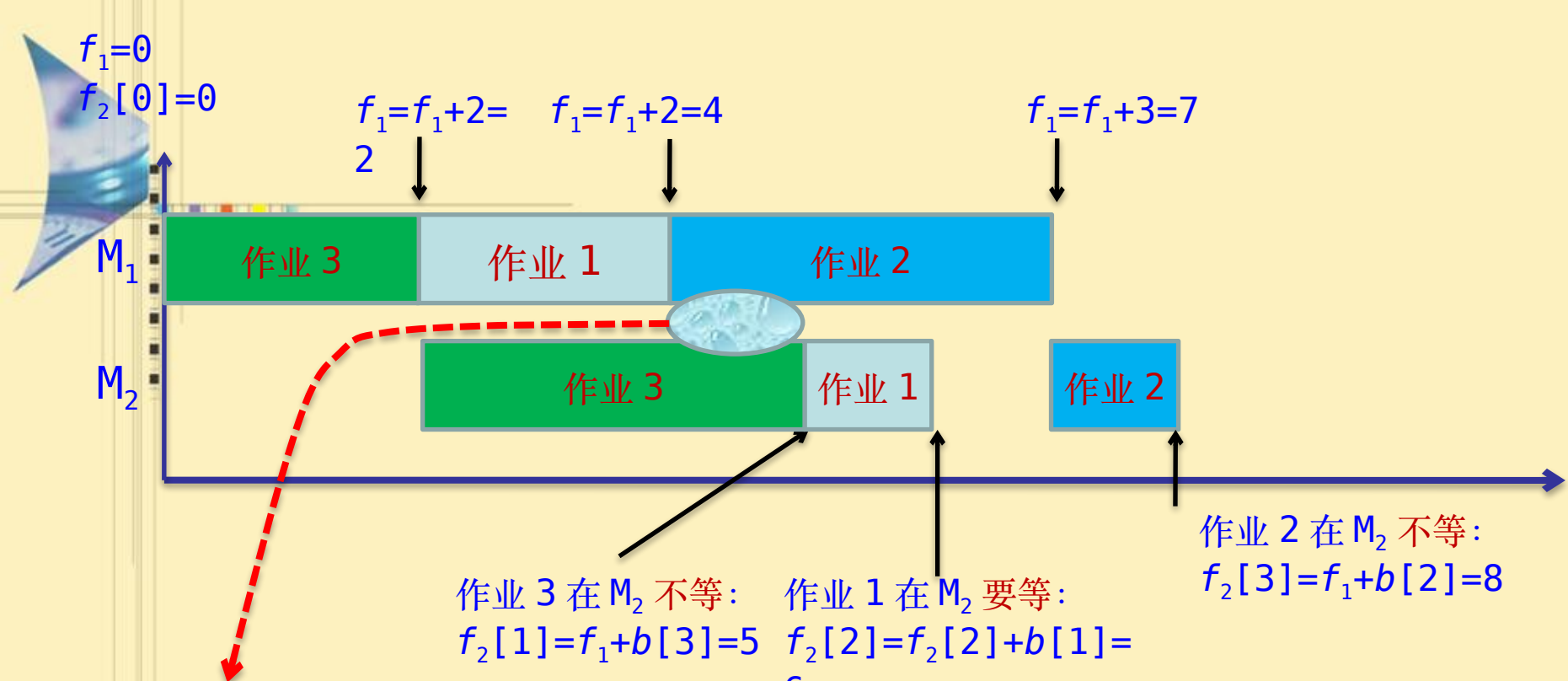
该调度方案的总时间：10

再看看另外一种调用方案 $x = (3, 1, 2)$ 。该调度方案的总时间计算:

作业编号	3	1	2
M_1 时间 a	2	2	3
M_2 时间 b	3	1	1



$x = (x[1], x[2], \dots, x[i], \dots, x[n])$
 ↑
 第 i 步执行的作业为 $x[i]$



第 i 步执行作业 $x[i]$ 在 M_2 上需要等待的条件: $f_2[i-1] > f_1$; 否则不需要等待

- $f_2[i-1] > f_1$: 需要等待 □ $f_2[i] = f_2[i-1] + b[x[i]]$
- $f_2[i-1] \leq f_1$: 不需等待 □ $f_2[i] = f_1 + b[x[i]]$

$f_1 += a[x[i]];$

$f_2[i] = \max(f_1, f_2[i-1]) + b[x[i]];$

算法实现

- 排列树递归回溯框架：在求一个方案的同时求其 $f_2[n]$ 时间！
- 剪枝：求出第 i 层的 $f_2[i]$ 即执行作业 $x[i]$ 后的总时间，若 $f_2[i] \geq \text{bestf}$ （ bestf 为当前求出的执行全部作业的最优总时间），就没有必要从该结点向下扩展了，让其成为死结点，也就是说仅仅扩展满足 $f_2[i] < \text{bestf}$ 的结点。

```

void dfs(int i)                                // 从第 i 层开始搜索
{
    if (i>n)                                    // 到达叶结点，产生一种调度方案
    {
        if (f2[n]<bestf)                        // 找到更优解
        {
            bestf=f2[n];
            for(int j=1; j<=n; j++)            // 复制解向量
                bestx[j] = x[j];
        }
    }
    else
    {
        for(int j=i; j<=n; j++)                // 没有到达叶结点，考虑可
        能的作业
        {
            swap(x[i],x[j]);
            f1 += a[x[i]];                      // 选择作业 x[i]，在 M1 上执行完的
            时间
            f2[i]=max(f1,f2[i-1])+b[x[i]];
            if (f2[i]<bestf)                    // 剪枝
                dfs(i+1);
            f1 -= a[x[i]];                      // 回溯
            swap(x[i],x[j]);
        }
    }
}

```

```
int n=4;           // 作业数
int a[MAX]={0,5,12,4,8}; //M1 上的执行时间，不用下标 0 的元素
int b[MAX]={0,6,2,14,7}; //M2 上的执行时间，不用下标 0 的元素
```



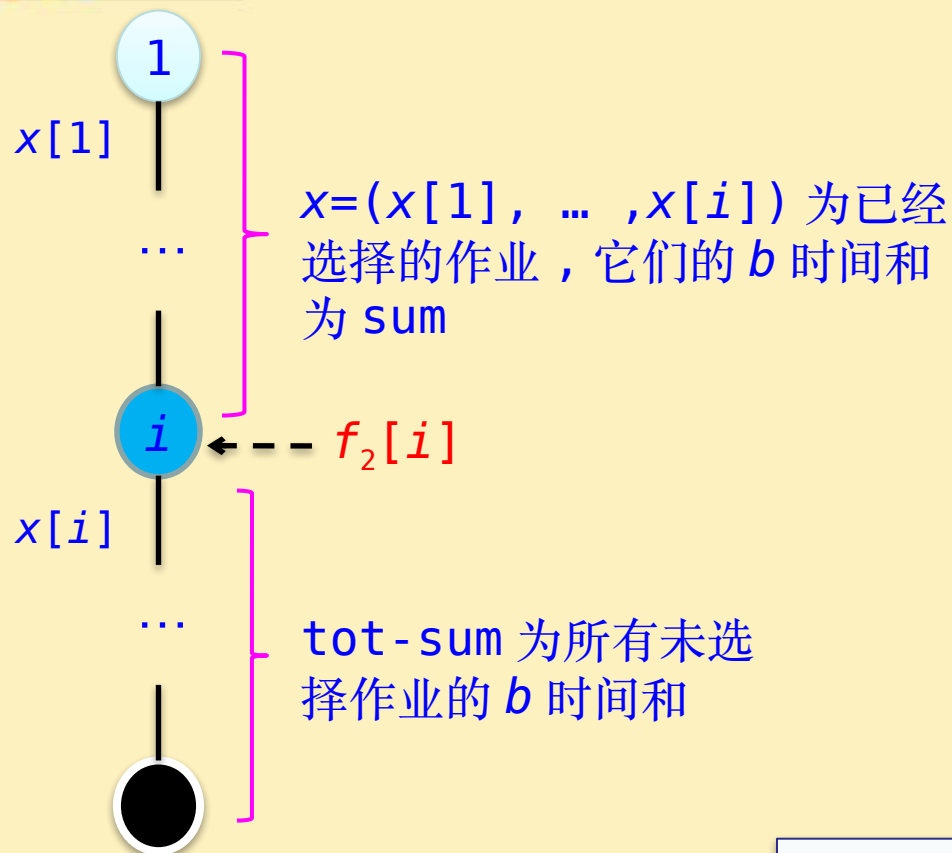
作业编号	1	2	3	4
M1 时间 <i>a</i>	5	12	4	8
M2 时间 <i>b</i>	6	2	14	7

求解结果：

最少时间：33，最优调度方案：3 1 4 2



进一步剪枝



该调度方案的时间和 $\geq f_2[i] + tot-sum$

■ 总时间 $\geq bound(i) \geq bestf \Rightarrow$ 剪枝

■ 仅仅扩展 $bound(i) < bestf$ 的结点

时间下界: $bound(i)$ 表示

剪枝函数

```
int bound(int i)
{
    int sum=0;
    for (int j=1;j<=i;j++)
        sum+=b[x[j]];
    return f2[i]+tot-sum;
}
```

// 求结点的下界值

// 扫描所有选择的作业

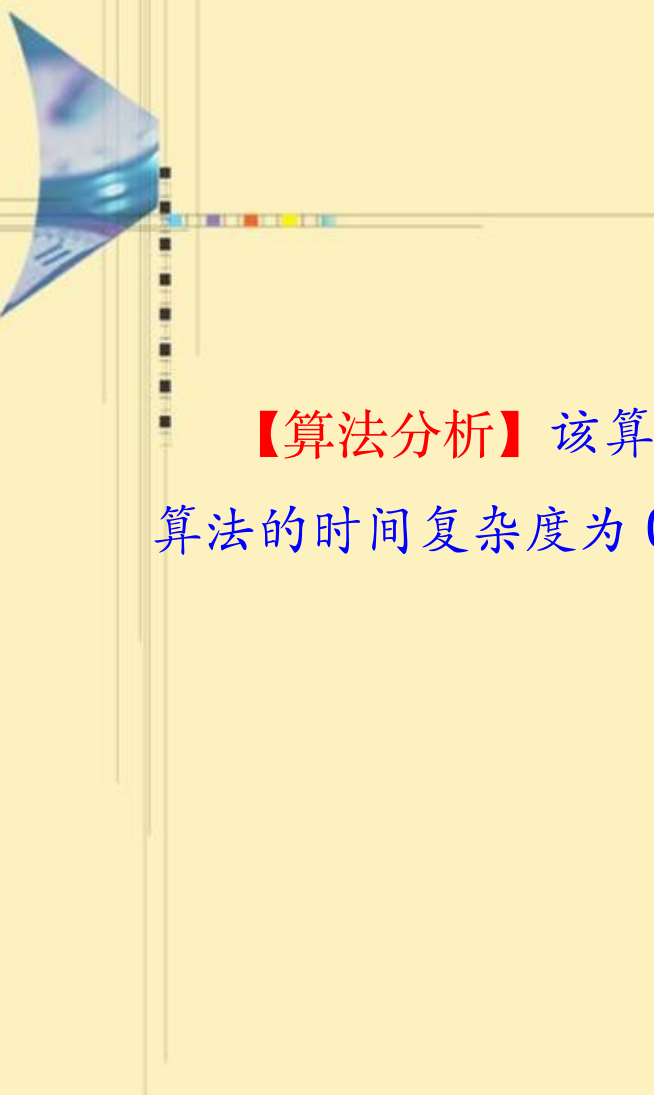
// 累计所有选择作业的 b 时间

// 全部 n 个作业的 b 时间和为 tot

```

void dfs(int i)                                // 从第 i 层开始搜索
{
    if (i>n)                                    // 到达叶结点，产生一种调度方案
    {
        if (f2[n]<bestf)                        // 找到更优解
        {
            bestf=f2[n];
            for(int j=1; j<=n; j++)            // 复制解向量
                bestx[j] = x[j];
        }
    }
    else
    {
        for(int j=i; j<=n; j++)                // 没有到达叶结点，考虑可
        能的作业
        {
            swap(x[i],x[j]);
            f1 += a[x[i]];                      // 选择作业 x[i]，在 M1 上执行完的
            时间
            f2[i]=max(f1,f2[i-1])+b[x[i]];
            if (bound(i)<bestf)                  // 剪枝
                dfs(i+1);
            f1 -= a[x[i]];                      // 回溯
            swap(x[i],x[j]);
        }
    }
}

```

【算法分析】该算法的解空间树是一棵高度为 n 的排列树，对应算法的时间复杂度为 $O(n!)$ 。

8.3.5 货郎担问题

例：货郎担问题， $S = \{1, 2, \dots, n\}$ ，当 $n=3$ 时， $S = \{1, 2, 3\}$ 。货郎担问题的解空间是：
 $\{(1, 1, 1), (1, 1, 2), (1, 1, 3), (1, 2, 1), (1, 2, 2), (1, 2, 3), \dots, (3, 3, 1), (3, 3, 2), (3, 3, 3)\}$

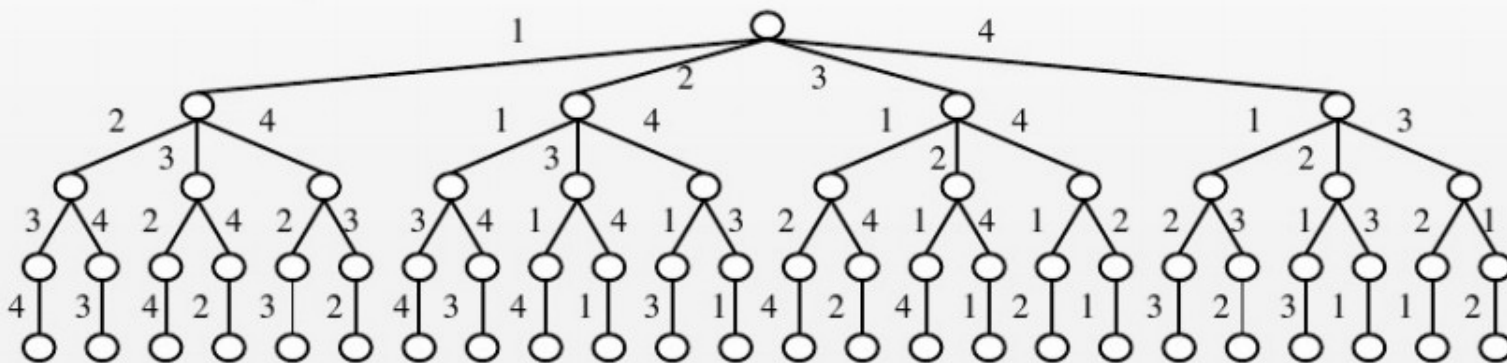
当输入规模为 n 时，它有 n^n 种可能的解。

考虑到约束方程 $x_i \neq x_j$ 。因此，货郎担问题的解空间压缩为：

$\{(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)\}$

当输入规模为 n 时，它有 $n!$ 种可能的解。

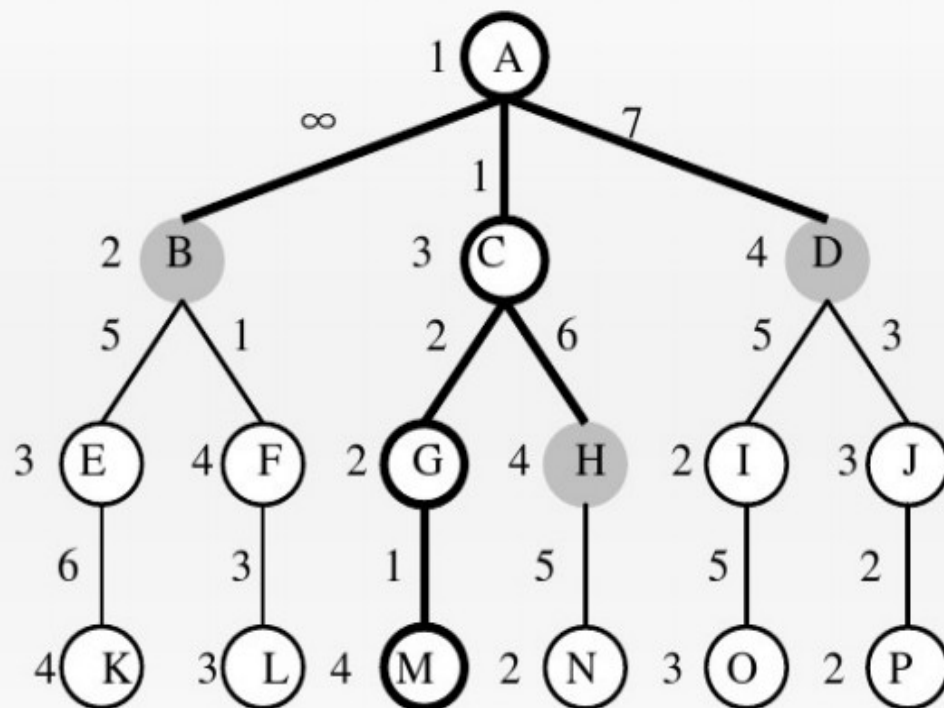
状态空间树：问题解空间的树形式表示
当 $n=4$ 时，货郎担问题的状态空间树。



8.3.5 货郎担问题

有4个顶点的货郎担问题，其费用矩阵如图7.3 所示，求从顶点1出发，最后回到顶点1的最短路线。

∞	∞	1	7
8	∞	5	1
7	2	∞	6
2	5	3	∞



8.3.6—— 限界函数、剪枝函数实践思考

- 给定一个数组，将该数组分成两部分 S1 和 S2，使得两部分个数的差最小，和的差最小。（绝对值）
- 给定一个数组，包含 $2N$ 个元素，将该数组分成大小相等的两部分 S1 和 S2，S1 和 S2 和的差最小。
- 给定一个数组，将该数组分成两部分 S1 和 S2，S1 和 S2 和相等。
- 给定一个数组，将该数组分成两部分 S1 和 S2，S1 和 S2 和的差最小。